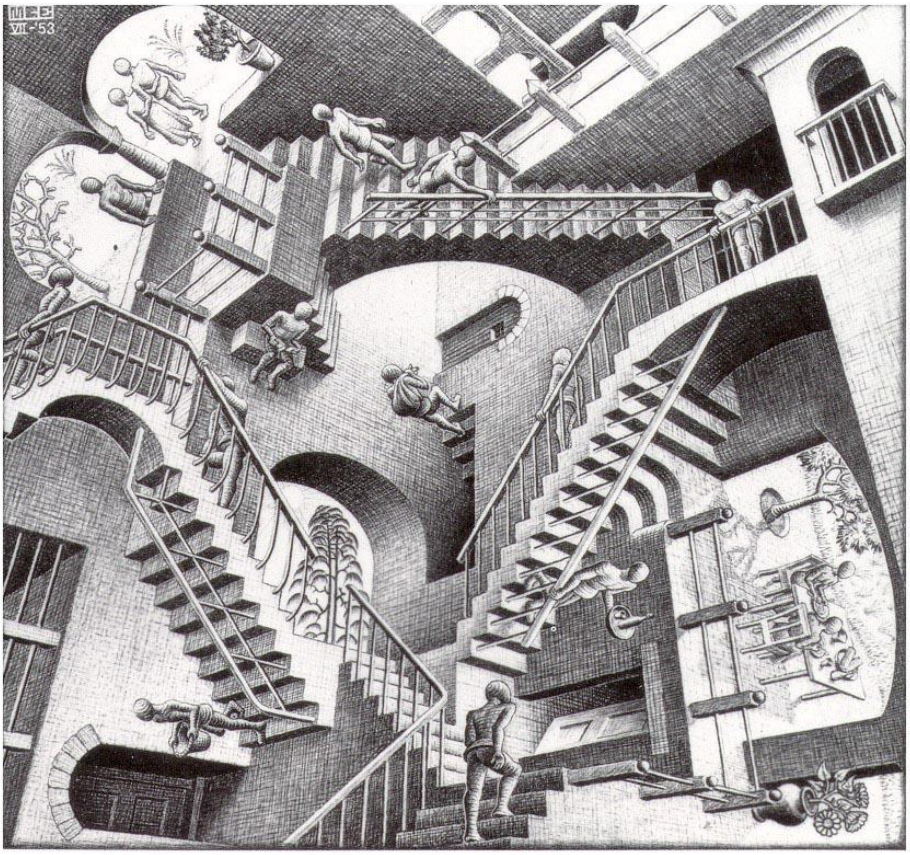
Análisis y Diseño de Software



Agosto 2017

Profesor: Patricio Letelier Torres

letelier@dsic.upv.es

Departamento de Sistemas Informáticos y Computación

Universidad Politécnica de Valencia

CONTENIDO

INTRODUCCIÓN AL MODELADO DE SI 2

INTRODUCCIÓN AL ANÁLISIS 7

ENFOQUE ESTRUCTURADO 9
- Modelado de Procesos 9
  - Explosión de un Proceso 10
  - Diccionario de Datos 11
  - Especificación de Proceso 12
  - ¿Qué técnica utilizar? 15
- Modelado de Datos 16
  - Diagramas Entidad-Relación (E/R) 16
  - Transformando un E/R en Tablas 24
  - Integrando Modelado de Procesos y Datos 29

ENFOQUE ORIENTADO A OBJETOS 30
- Breve Tour por UML 32
  - Paquetes en UML 32
  - Diagrama de Casos de Uso 33
  - Diagrama de Secuencia 36
  - Diagrama de Colaboración 37
  - Diagrama de Clases 38
  - Diagrama de Estados 42
  - Diagrama de Actividad 43
  - Diagrama Componentes 44
  - Diagrama de Despliegue 45
  - Comentarios respecto de los Diagramas de UML 46
- Modelado de Datos en el enfoque OO 47
- Modelado de Requisitos 48

MODELADO DE SI: REFLEXIONES FINALES 49

INTRODUCCIÓN AL MODELADO DE SI

Antes de entrar en materia respecto al modelado de sistemas de información conviene recordar que nuestro contexto es el negocio de producción de software, es decir, cualquier mejora que queramos abordar desde el punto de vista del modelado deberá estar enmarcada en los cuatro objetivos esenciales de cualquier negocio (ilustrados en la Figura 1): aumentar la calidad del producto, disminuir los costos de producción, aumentar la productividad y aumentar la satisfacción de nuestros clientes (y también de los desarrolladores del producto).

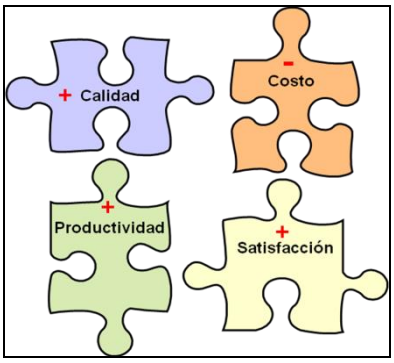


FIGURA 1: OBJETIVOS EN PRODUCCIÓN DE SOFTWARE

Claves en Desarrollo de Software. La Figura 2 ilustra los elementos claves para el éxito de un proyecto de software[1].

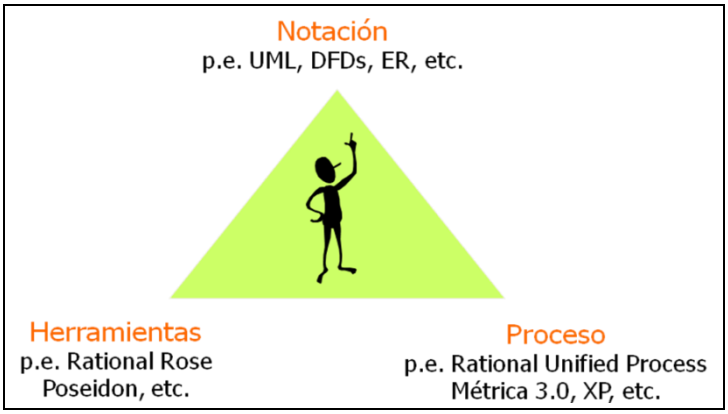


FIGURA 2: CLAVES EN DESARROLLO DE SOFTWARE

[1] Figura "Triangle for Success" adaptada desde "Visual Modeling with Rational Rose and UML" de Terry Quatrani

Como es de natural, las personas (equipo de desarrollo y cliente) son el factor más determinante, muchas veces mayormente gracias a la capacidad o el esfuerzo extraordinario de las personas se logra llevar a buen fin un proyecto de desarrollo de software. Pero pasando al terreno puramente técnico, si un equipo de desarrollo se plantease mejorar su desempeño, podría potenciar tres aspectos: notación, herramientas y/o proceso.

La notación se refiere a la forma en la cual especificamos el producto. Esto incluye desde lenguajes de programación hasta notaciones gráficas para elaborar diagramas. Mientras más sofisticadas sean las notaciones más se agradece contar con herramientas específicas que faciliten el uso de dichas notaciones. Un diagrama UML puede hacerse con papel y lápiz, pero es indudable (especialmente de cara al mantenimiento del diagrama) que es mucho más efectivo hacerlo con una herramienta, en este caso un editor gráfico para diagramas UML. Finalmente, de poco sirve dominar una notación y contar con potentes herramientas si no sabemos contextualizar esto en un proyecto de desarrollo, es decir, saber quién (que rol o persona) tiene que hacer qué (qué artefactos), cuándo (basado en un plan) y cómo (definición de actividades usando notación y herramientas). Estos tres elementos: roles, artefactos y actividades son orquestados por un proceso de desarrollo (o comúnmente llamada metodología de desarrollo).

¿Por qué modelar el Software? Un Modelo es una representación del software, lo cual implica un cierto grado de abstracción respecto al producto final. El modelado de software facilita:

- Comunicación de ideas
- Evaluar alternativas
- Aproximación gradual al producto, mediante un proceso colaborativo de descubrimiento
- “Visualizar” el producto, antes de construirlo

Un **Modelo** captura una vista de un sistema del mundo real. Es una abstracción de dicho sistema, considerando un cierto propósito. Así, el modelo describe completamente aquellos aspectos del sistema que son relevantes al propósito del modelo, y a un apropiado nivel de detalle.

Un **Diagrama** es una representación gráfica de una colección de elementos de un modelo, a menudo dibujada como un grafo con vértices conectados por arcos. Así, un modelo se representa a través de 1 o más diagramas. Un proceso de desarrollo de software debe ofrecer un conjunto de modelos que permitan expresar el producto desde cada una de las perspectivas de interés.

El código fuente del sistema es el modelo más detallado del sistema (y además, ¡es ejecutable!). Sin embargo, se requieren otros modelos más abstractos. Cada modelo es completo desde su punto de vista del sistema, sin embargo, existen relaciones de **Trazabilidad** entre los diferentes modelos. En la Figura 3 podríamos tener a la izquierda especificaciones de requisitos y a la derecha piezas de código. Todos los requisitos deberían estar soportados por algunas piezas de código y por otra parte, no deberían existir piezas de código que no tengan una justificación en los requisitos. El Análisis de Impacto de un cambio en el sistema es otra de las motivaciones para mantener actualizada la trazabilidad, así podemos tener información respecto de los artefactos que podrían verse afectados por un cambio.

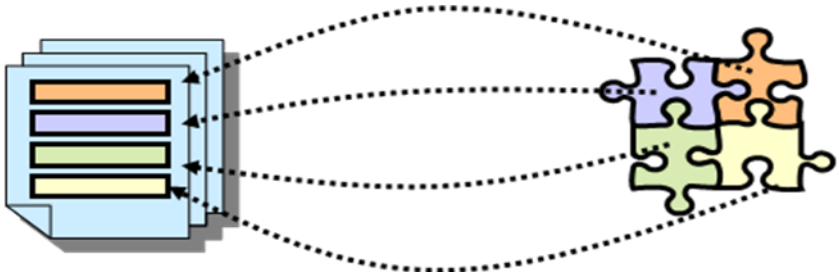


FIGURA 3: RELACIONES DE TRAZABILIDAD ENTRE ARTEFACTOS

La trazabilidad es vital para conseguir el éxito del proyecto. Sin embargo, puede llegar a ser muy costoso mantener actualizadas las relaciones de trazabilidad entre los artefactos que se generan durante el proyecto. Si intentáramos ser radicales, y con el propósito de reducir al máximo el esfuerzo asociado a mantener trazabilidad, podríamos decidir tener sólo el código como artefacto generado, pero, aunque sea de forma explícita, al menos tendremos que tener requisitos.

¿Cuánto modelar? Depende de la envergadura y/o complejidad del producto y debe además estar acorde con la explotación que se haga de los modelos. Haciendo una comparación con la construcción de casas y edificios (ver Figura 4 continuación), estaremos de acuerdo en que respecto del ámbito de notaciones, de herramientas y de proceso no es lo mismo construir una casa para nuestra mascota que construir una casa familiar, ni mucho menos que construir un edificio (en la Figura 4, a la derecha, el Burj Khalifa, el rascacielos más alto del mundo).

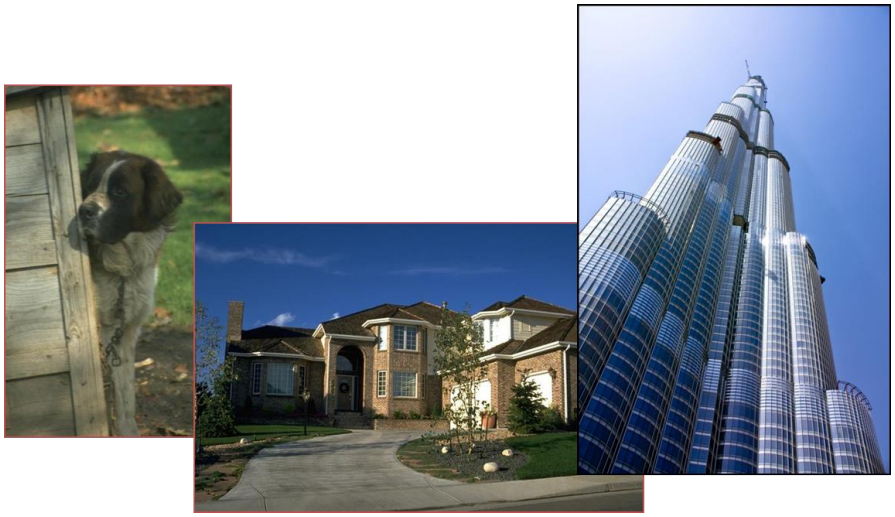


FIGURA 4: PROYECTOS DE DIFERENTE ENVERGADURA/COMPLEJIDAD

¿Qué es un producto/proyecto de software pequeño, mediano o grande? Podemos, por ejemplo, catalogar basándonos en: líneas de código producidas, tiempo de desarrollo, número de participantes en el proyecto, financiación de proyecto, número y complejidad de los requisitos, etc. Así, un proyecto llevado a cabo por un equipo de 3 desarrolladores durante 4 meses sería un proyecto pequeño (generando unas pocas decenas de miles de líneas de código), un proyecto con un equipo de 10 desarrolladores trabajando durante 8 meses podría catalogarse de mediano (pensando que generaría unos cientos de miles de líneas de código. Un proyecto de cientos de desarrolladores y años de desarrollo (generando millones de líneas de código) podría considerarse grande (por ejemplo, el desarrollo de un sistema operativo).

Obviamente debe ser el contexto de desarrollo (envergadura/complejidad del proyecto) el que determina la configuración adecuada del proceso y los recursos necesarios. Existen propuestas radicales que promueven un proceso/modelado más “light”, tales como: Extreme Programming (Kent Beck) y Agile Modeling (Scott Ambler). Sin embargo, en muchos proyectos es difícil eludir un proceso y modelado más rigurosos, debido por ejemplo a relaciones contractuales y la gran envergadura o complejidad del proyecto.

Otra cuestión importante en cuanto al grado de modelado es el aprovechamiento que se hace de dicho modelado. Debe obtenerse un ratio interesante entre el esfuerzo y tiempo gastado en modelar respecto del aprovechamiento que se hace de dichos modelos. El nivel de aprovechamiento de los modelos se ilustra en la Figura 5.

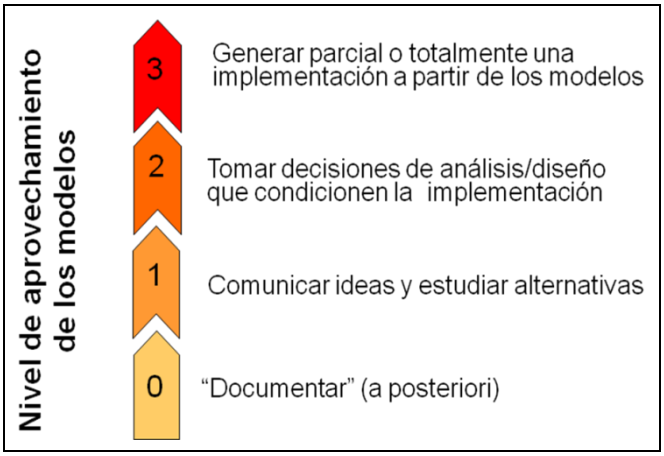


FIGURA 5: NIVELES DE APROVECHAMIENTO DEL MODELADO

Lo menos rentable es utilizar los modelos simplemente para documentar después de construir el producto software (nivel 0). Si al menos los modelos se utilizan para comunicar ideas o estudiar alternativas durante el proyecto se consigue un mejor aprovechamiento, y de paso se puede adelantar la documentación del sistema. Cuando además el uso de los modelos conlleva la toma de decisiones de análisis/diseño que condicionan la implementación estaremos en un buen nivel de aprovechamiento (y conseguiremos además las ventajas de los niveles inferiores). El máximo aprovechamiento se conseguirá cuando los modelos generen de forma automática un producto (aunque no sea 100% el producto final), esta es la tendencia en ingeniería de software "generación automática de código a partir de modelos".

INTRODUCCIÓN AL ANÁLISIS

¿Qué es Análisis? Definición clásica: Una especificación de análisis es independiente de la plataforma de implementación. Otras frases usuales: “Análisis es establecer el qué, mientras que diseño es establecer el cómo”.

El análisis clásico actualmente se debería separar en Requisitos y Análisis. Una especificación de requisitos provee una vista de usuario del sistema, sin entrar en la especificación interna del producto. Una especificación de análisis describe el producto globalmente en cuanto a su arquitectura interna. En la iniciativa de generación de código a partir de modelos, específicamente en la propuesta MDA (Model-Driven Architecture), se establecen los modelos CIM (Computer Independent Model) y PIM (Platform Independent Model) para distinguir entre un nivel de requisitos y de análisis. En un enfoque Orientado a Objetos normalmente se encadenan Análisis y Diseño sin enfatizar sus diferencias. No sucede lo mismo en el Enfoque Estructurado pues se utilizan diferentes notaciones para análisis y para diseño.

Enfoques para el Desarrollo de Software. La Figura 6 resume el estado del arte en cuanto a notaciones y enfoques para el modelado de sistemas de información. Para facilitar la explicación se ha representado un proceso de desarrollo secuencial pero no tiene por qué ser así. También se ha simplificado el proceso restringiéndolo a actividades de Requisitos, Análisis, Diseño e Implementación, debido a que en las 3 primeras suele concentrarse la mayor parte del modelado de un sistema de información. También, suponemos que la implementación (común a cualquier enfoque) se hace con programación en Forms o Web, usando un gestor de Bases de Datos Relacional.

Para un producto software siempre tendremos que especificar sus dos facetas (procesos y datos), en cualquier nivel de abstracción, incluso en la implementación, donde tendremos un código fuente en un lenguaje de programación que accederá a una base de datos. Como se observa en la figura en el Enfoque Estructurado se utilizan DFDs y Diagramas E/R para especificar requisitos y para el análisis, pero los modelos representados con dichos diagramas deben convertirse en Diagramas de Estructura y Diagramas de Tablas (Modelo lógico relacional) en el momento de diseño. En el Enfoque Orientado a Objetos, los diagramas de UML (y en especial el Diagrama de Clases) son la notación utilizada tanto para análisis como para diseño. Sin embargo, cuando la implementación se hace en una Base de Datos Relacional debería, independiente del enfoque deberíamos contar con el diseño relacional de dicha base de datos. Esto constituye una especie de parche en el caso del Enfoque Orientado a Objetos. De hecho, esta impedancia de la orientación a objetos respecto de las Bases de Datos Relacionales se suele hacer evidente en la implementación (esto se comenta más adelante en este documento).

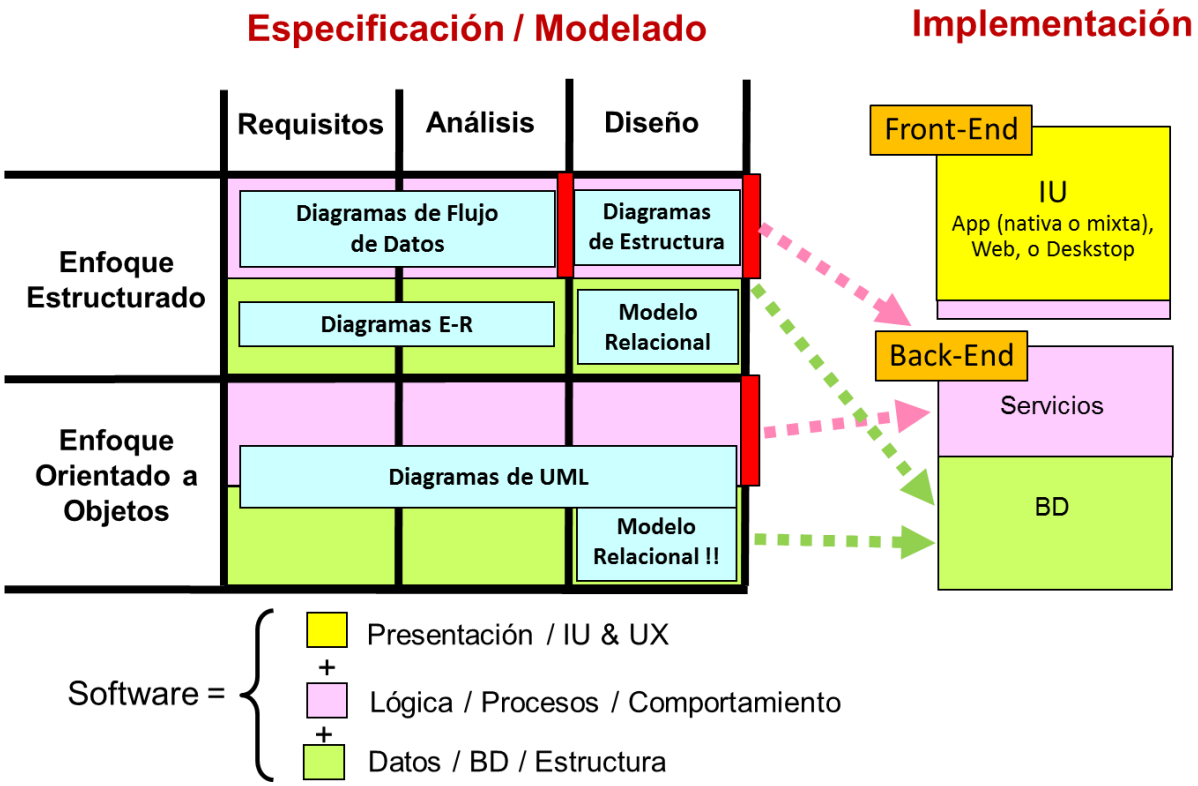


FIGURA 6. MODELADO DE SOFTWARE EN ENFOQUES ESTRUCTURADO Y ORIENTADO A OBJETOS

ENFOQUE ESTRUCTURADO

El Enfoque Estructurado utiliza en el Análisis Estructurado las siguientes notaciones:

- Diagramas de Flujo de Datos (DFD)
- Diccionario de Datos (DD)
- Especificaciones de Procesos
- Diagramas Entidad-Relación (ER)
- Diagramas de Transición de Estados (DTE)

A continuación, veremos resumidamente cada una de estas notaciones. Sólo dejaremos pendiente la explicación de los Diagramas de Transición de Estados, pues se recomienda utilizar para ello la notación de Diagramas de Estado propuesta en UML, los cuales son explicados en el apartado de modelado orientado a objetos.

Modelado de Procesos

El modelado de procesos en el Análisis Estructurado se hace utilizando Diagramas de Flujo de Datos (DFDs). Una de las principales ventajas de los DFDs es su sencillez. La Figura 7 muestra un ejemplo de DFD correspondiente a una parte del análisis de un sistema. Como se observa, es fácil entender que los pedidos recibidos de los clientes se verifican para posteriormente armar ordenes de compra a los editores.

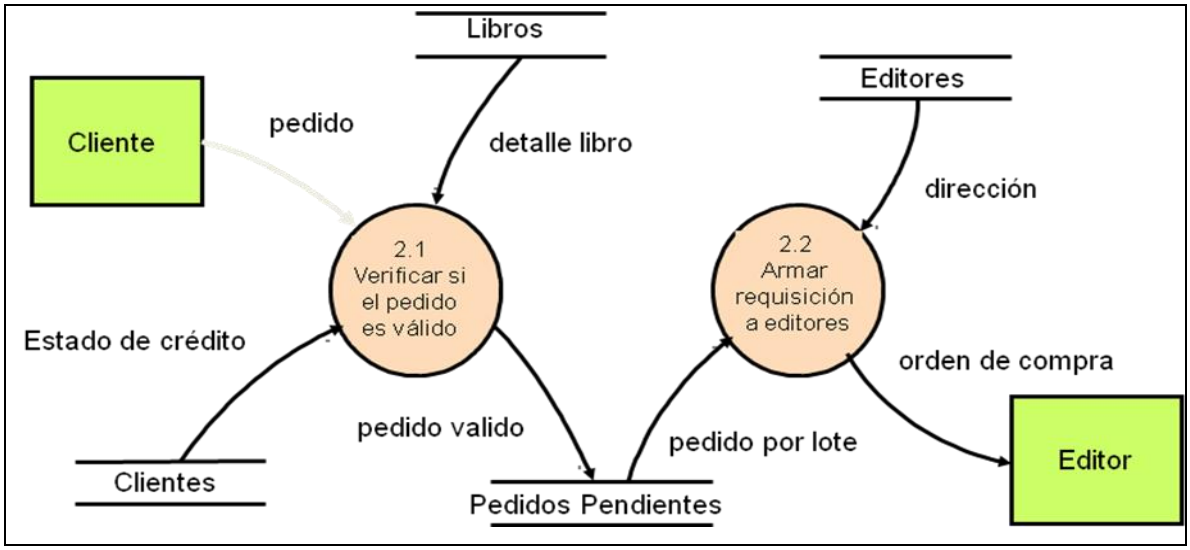


FIGURA 7: EJEMPLO DE DIAGRAMA DE FLUJO DE DATOS

En un DFD los procesos representan a algo o alguien que transforma y/o manipula flujos de datos. La

Figura 8 muestra la notación utilizada para un proceso.

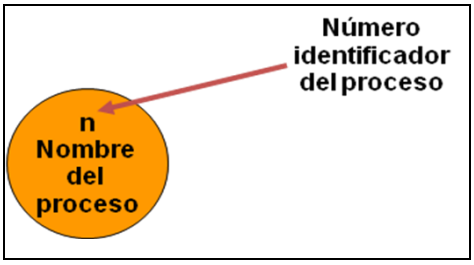


FIGURA 8: NOTACIÓN PARA PROCESOS

Algunos ejemplos de procesos se muestran en la Figura 9.

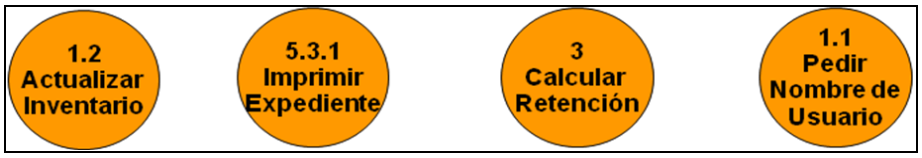


FIGURA 9: EJEMPLOS DE PROCESOS

Los flujos de datos representan movimientos de información dentro del sistema. Así, un proceso transforma flujos de entrada en flujos de salida.

Explosión de un Proceso

Consiste en desagregar un proceso padre en un nuevo DFD de mayor detalle. Normas a seguir al explosionar un proceso:

- Numeración: Al explosionar el proceso “n”, se numerarán los procesos hijos como n.1, n.2, ...
- DFD Balanceado: Todos los flujos que entran o salen del proceso padre deberán entrar y salir del conjunto de procesos hijos

La Figura 10 ilustra la explosión del proceso 0 en el DFD de Contexto, generando el DFD 0, en el cual la explosión del proceso 2 genera el DFD 2.

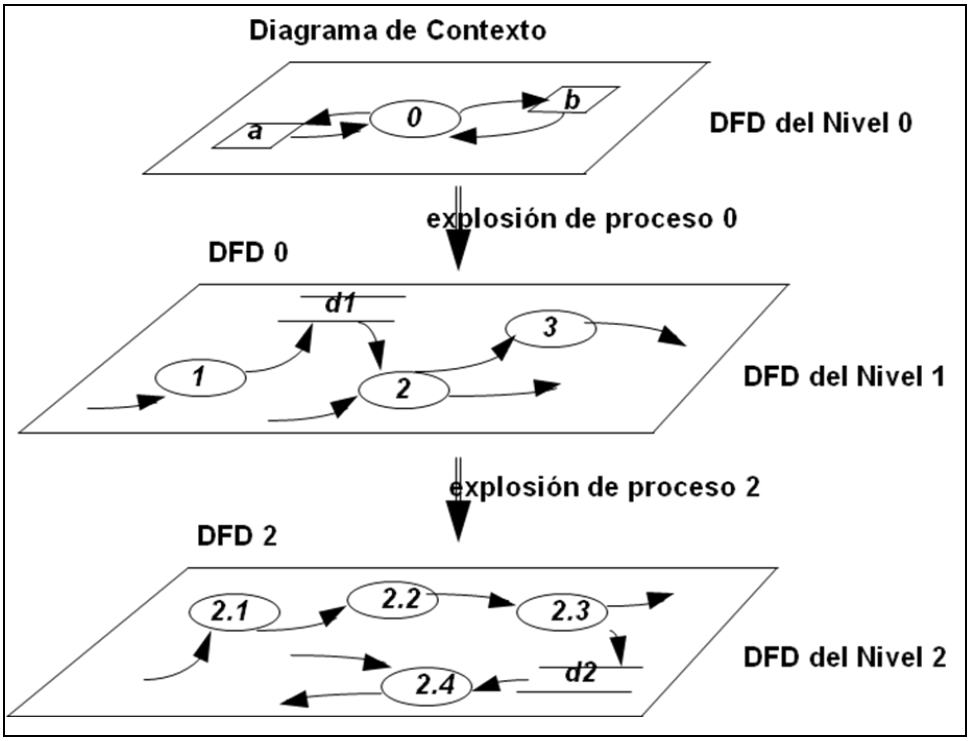


FIGURA 10: EXPLOSIÓN DE PROCESOS

Diccionario de Datos

El Diccionario de Datos contiene la descripción de cada dato. Existirá al menos una entrada por cada flujo o almacén que aparezca en los DFDs. Cada estructura de datos se describe hasta el nivel elemental. Cada dato debería tener una definición que incluya:

- Un comentario que explique su significado
- Composición, si no es un dato elemental
- Valores posibles

Ejemplo:

```
Nombre =  título_cortesía+
          primer_nombre+
          (segundo_nombre)+
          apellido_paterno+
          apellido_materno
título_cortesía = [Sr| Sra.|Don|Doña]
primer_nombre = {caracter_permitido}
segundo_nombre = {caracter_permitido}
apellido_paterno = {caracter_permitido}
apellido_materno = {caracter_permitido}
caracter_permitido = [A-Z|a-z|0-9|'|-| |]
```

Especificación de Proceso

Son descripciones de la lógica interna de los procesos de último nivel. Definen qué debe hacerse para transformar las entradas en salidas. Para especificar u proceso pueden utilizarse de forma alternativa o en conjunto las siguientes notaciones:

- Descripción narrativa
- Interfaz-Función
- Lenguaje Estructurado o pseudocódigo
- Árboles de decisión
- Tablas de decisión
- Diagramas de flujo
- Diagramas de Nassi-Schneiderman

Por defecto, la forma de especificar un proceso es una descripción narrativa en lenguaje natural

Pseudocódigo

Una especificación en pseudocódigo tiene como principal ventaja su precisión. A continuación se muestra un ejemplo de pseudocódigo que correspondería al proceso de la Figura 11.

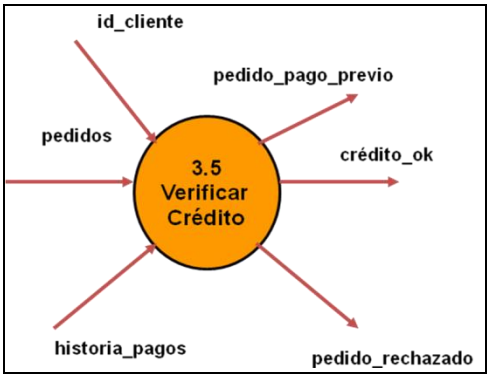


FIGURA 11: PROCESO VERIFICAR CRÉDITO

```
LEER pedidos con id_cliente
SI cliente es nuevo
     EMITIR pedido_pago_previo
SINO
     SI cliente es frecuente (*promedio de dos pedidos
     mensuales*)
          OBTENER balance desde historia_pagos
          SI balance esta vencido hace más de dos meses
               EMITIR pedido_rechazado
          SINO
               EMITIR credito_ok
          FIN_SI
      SINO
          EMITIR pedido_pago_previo
      FIN_SI
FIN_SI
```

Interfaz Función

Una descripción interfaz-función para un proceso consiste simplemente en indicar las entradas y salidas del proceso junto con una breve descripción narrativa de la función del proceso. A continuación se muestra un ejemplo asociado al proceso "Seleccionar asiento de pasajero" en un supuesto sistema de reserva.

PROCESO : SELECCIONAR ASIENTO DE PASAJERO

ENTRADAS: PREFERENCIA_ASIENTO_PONDERADA

SALIDAS : ASIENTO_SELECCIONADO

PREFERENCIA_DISPONIBLE

FUNCIÓN : Seleccionar un asiento para un pasajero conside-rando que sus preferencias de ubicación sean lo más cercanas (ponderadamente) al asiento elegido

Árbol de Decisión

Un Árbol de Decisión es recomendado cuando el número de acciones es reducido y no son posibles todas las combinaciones. La Figura 12 muestra un ejemplo de árbol de decisión donde se observa la ramificación producida por las condiciones y su evaluación, y a la derecha las acciones que deberían aplicarse.

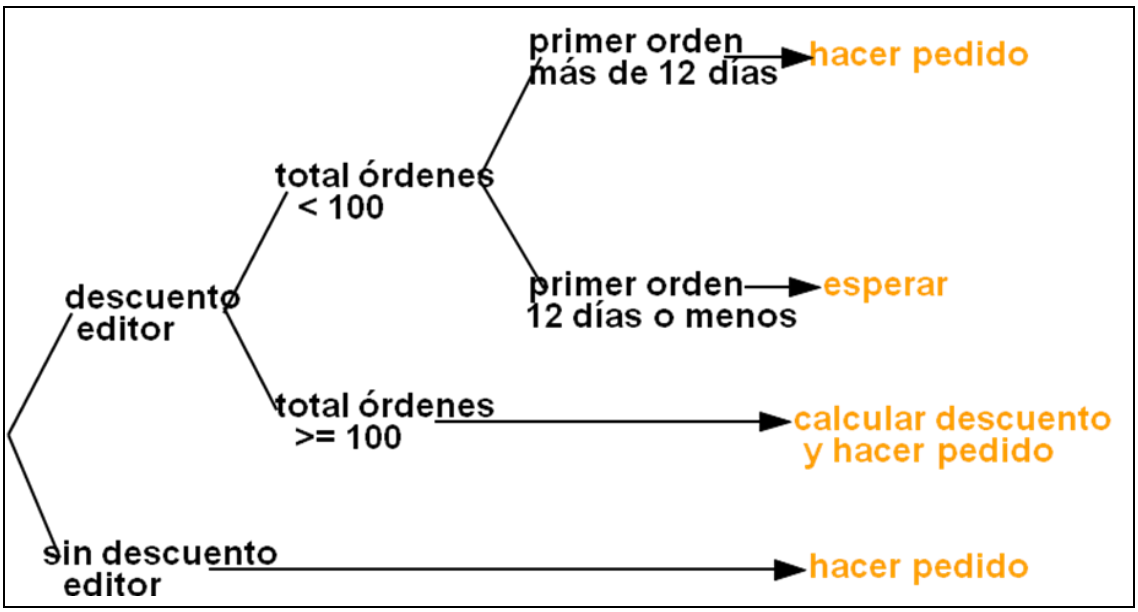


FIGURA 12: EJEMPLO DE ÁRBOL DE DECISIÓN

Tabla de Decisión

Una Tabla de Decisión se recomienda cuando existen muchas combinaciones y se quieren revisar todas en detalle. El siguiente ejemplo muestra cómo basándose en características de un supuesto paciente, se le asigna un cierto tratamiento. Podemos ver que 3 condiciones binarias producen 8 combinaciones posibles, y para cada una de ellas se establece la acción (tratamiento) aplicable.

Combinaciones posibles

		1	2	3	4	5	6	7	8
Condiciones	edad > 21	Y	Y	Y	Y	N	N	N	N
	sexo	V	V	M	M	V	V	M	M
	peso > 60	Y	N	Y	N	Y	N	Y	N
Acciones	Medicamento 1	X				X			X
	Medicamento 2		X			X			
	Medicamento 3			X			X		X
	Ninguno				X			X	

Asignación de acciones

FIGURA 13: EJEMPLO DE TABLA DE DECISIÓN

Diagrama de Flujo

Los Diagramas de Flujo son una de las primeras notaciones de modelado para algoritmos y código. La muestra los constructores básicos a partir de los cuales, en combinación se pueden especificar flujos de control.

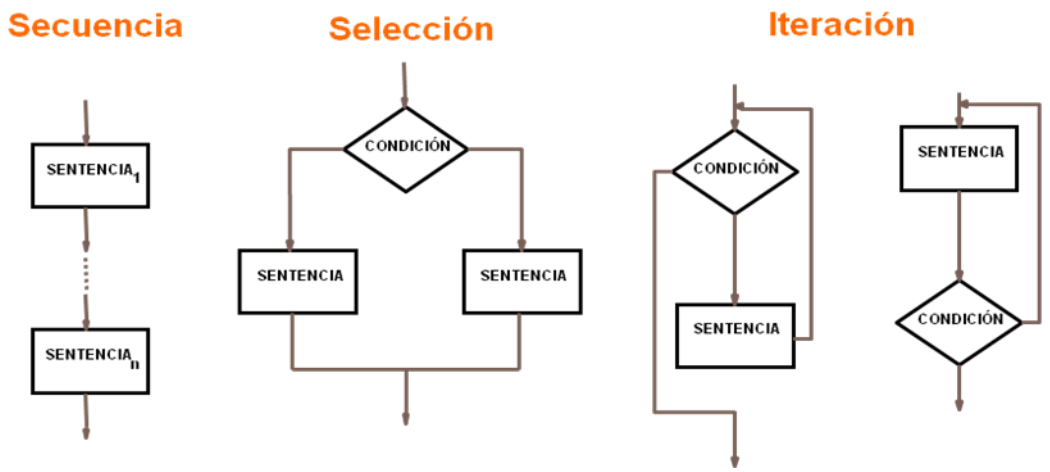
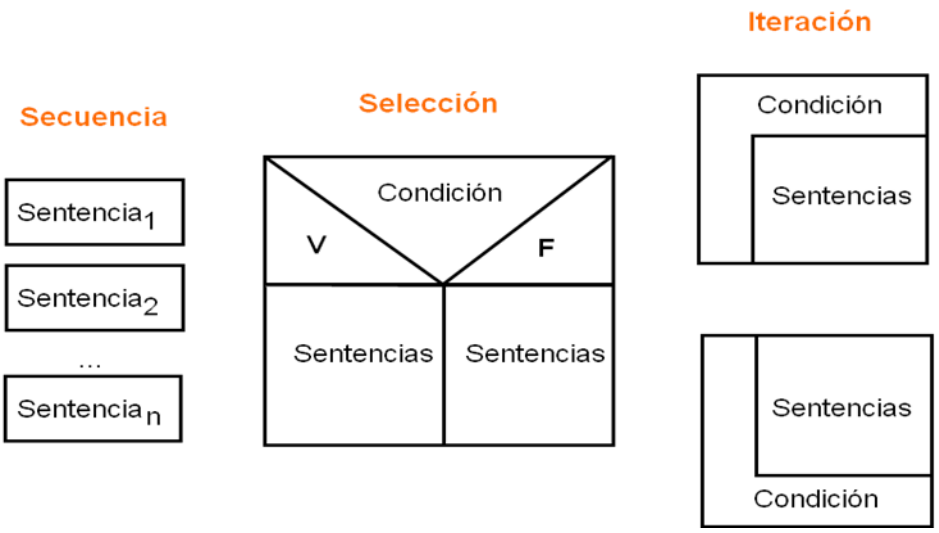


FIGURA 14: CONSTRUCTORES BÁSICOS DE UN DIAGRAMA DE FLUJO

Diagramas de Nassi-Schneiderman

Estos diagramas se basan en bloques que representan los constructores estructurados de secuencia, selección e iteración, y que pueden componerse en secuencia o anidarse.



¿Qué técnica utilizar?

Las técnicas presentadas para especificar procesos pueden usarse de forma independiente o en conjunto. Dependiendo de las características del proceso que se desea especificar, una técnica puede ser más conveniente que otra, o incluso varias técnicas pueden resultar complementarias si el proceso lo requiere.

En primer lugar, hay que tener presente que cuando una especificación de proceso debe ser validada por el cliente, no podemos obligarlo a entrar en cuestiones técnicas que dificulten su legibilidad. Esto es un gran obstáculo puesto que en algunos casos puede que sólo una especificación en lenguaje natural (narrativa) resulte apropiada. Así pues, una especificación Interfaz-Función es lo más sencillo para mostrar a un cliente. Sin embargo, cuando se requiere precisión y nuestro cliente puede entender una especificación más sofisticada sería mejor utilizar otra técnica. El pseudocódigo es lo más conveniente desde la perspectiva de los desarrolladores pues incluso puede servir como una versión preliminar del código que se debe implementar. Sin embargo, debería mantenerse en un nivel mucho más abstracto para que realmente aporte al entendimiento global del proceso. No tiene sentido hacer un pseudocódigo de 1000 líneas que finalmente se transformara en 1000 líneas de código fuente. Los Diagramas de Flujo y los Diagramas de Nassi-Schneiderman, respecto del pseudocódigo, tienen además la ventaja de ser notaciones gráficas. Una característica interesante de los Diagramas de Nassi Scheiderman es que no permiten “goto” es decir, no permiten programación NO estructurada. En los Diagramas de Flujo podemos introducir todos los saltos que se desee, lo cual puede generar algoritmos difíciles de entender. Desgraciadamente casi no existe soporte de herramientas para los Diagramas de Nassi-Scheiderman. Las Tablas y Árboles de Decisión son muy útiles cuando hay muchas acciones dependientes de una combinación de condiciones. Esta situación en pseudocódigo puede

resultar difícil de entender y especificar mediante muchos niveles de selecciones anidadas. Si se quieren explorar todas las combinaciones de valores de condiciones es preferible una Tabla de Decisión, sino es más apropiado un Árbol de Decisión. Tanto Tablas como Árboles de Decisión no suelen ser una solución para describir todo el proceso, sino más bien una parte concreta, con lo cual, normalmente se utilizan en conjunto con otra técnica.

Más adelante, cuando se presente el enfoque orientado a objetos conviene tener en cuenta que todas estas técnicas para especificar procesos pueden también ser utilizadas para especificar operaciones de clases.

Modelado de Datos

El modelado de datos en el enfoque estructurado se puede realizar de forma paralela al modelado de procesos. Sin embargo, en aplicaciones de gestión, suele ser conveniente adelantar un poco el modelado de datos respecto del modelado de procesos. La justificación a esto es que un sistema de gestión suele centrarse en la manipulación relativamente sencilla de información variada, con lo cual el modelo de datos determina en gran medida la arquitectura interna del producto. Esto no debe mal entenderse en el sentido extremo en el cual, por ejemplo, si un sistema tiene 20 tablas en su base de datos, entonces se ofrecerá un sistema que manipule mediante las operaciones alta, baja, modificación y consulta cada una de dichas tablas. El diseño de interfaces adaptadas a las necesidades de usuario (la metáfora visual y forma más adecuada de trabajo) y todas las reglas de negocio que rigen el acceso y tratamiento de la información hacen que dicha propuesta de solución extrema resulte totalmente rudimentaria.

Diagramas Entidad-Relación (E/R)

Los Diagramas E/R son quizás la notación de modelado más exitosa y usada en la práctica. Esta afirmación puede parecer exagerada, puesto que se podría objetar que los Diagramas de Clases son aún más populares. Pero lo cierto es que en la mayoría de los casos los Diagramas de Clases son utilizados como si fuesen Diagramas E/R. Ya comentaremos esto más adelante.

Entidad: Un objeto del mundo real relevante para el sistema modelado, que puede ser distinguido y posiblemente tener propiedades (atributos). Una entidad tendrá un valor (o varios) para cada uno de sus atributos.

Los atributos pueden ser:

- Simples / Compuestos
- Univaluados / Multivaluados
- Derivados / No Derivados
- Valor NULO permitido / Valor NULO no permitido

Estos tipos de atributos se representan gráficamente tal como se muestra en la Figura 15.

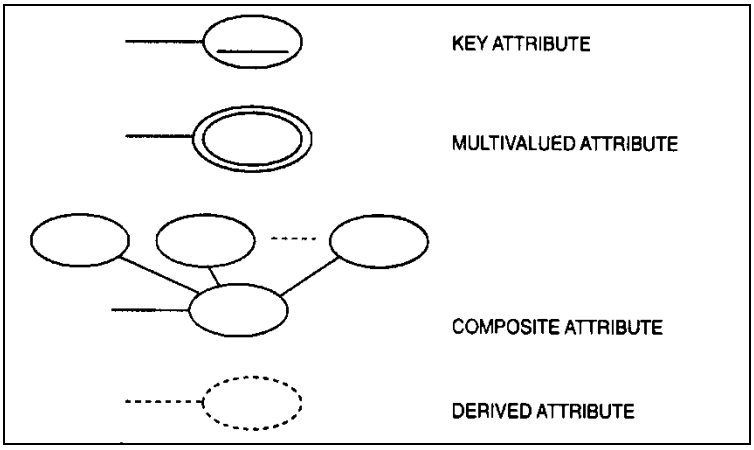


FIGURA 15: REPRESENTACIÓN GRÁFICA DE LOS ATRIBUTOS

Tipo de Entidad: Define un conjunto de entidades que tienen los mismos atributos. La Figura 16 ilustra cómo se representan los tipos de entidades.

Atributos Clave: Atributos de una Entidad que permiten identificarla unívocamente.

Relación: Referencia entre entidades. Una entidad NO puede tener como atributo a otra entidad, esta situación se interpreta como la necesidad de existencia de una relación entre ambas entidades. Una relación puede tener atributos.

Tipo de Relación: Define un conjunto de relaciones entre tipos de entidades. La Figura 16 ilustra cómo se representan los tipos de relaciones.

Aunque lo correcto sería decir que modelamos Tipos de Entidad y Tipos de Relación en un Diagrama E/R, usualmente se les llama simplemente Entidades y Relaciones. Con lo cual usaremos esta denominación abreviada.

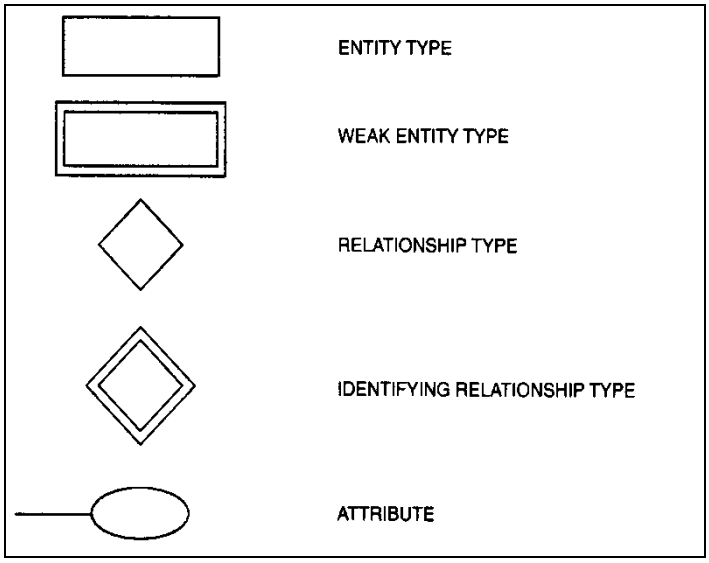


FIGURA 16: REPRESENTACIÓN GRÁFICA DE ENTIDADES, RELACIONES, Y ATRIBUTO SIMPLE

Grado de la relación: Es el número de tipos de entidades que participan en un tipo de relación. Según su grado, los tipos de relaciones se denominan binarias reflexivas, binarias, ternarias, etc. La Figura 17 ilustra relaciones de diferentes grados y la Figura 18 muestra ejemplos de cada tipo.

Nombres de Rol: Es el nombre asociado al papel que desempeña un tipo de entidad en un tipo de relación. En el caso de relaciones binarias reflexivas o cuando hay más de una relación entre dos entidades, es importante especificar el nombre del rol.

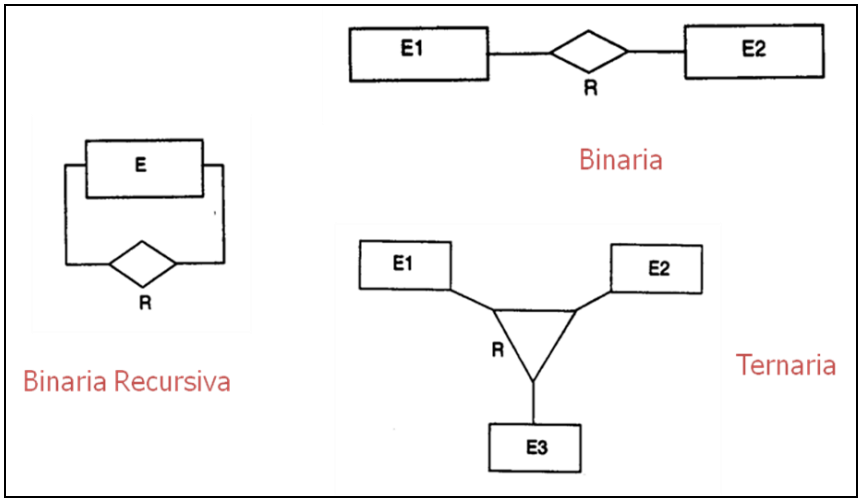


FIGURA 17: RELACIONES DE DIFERENTES GRADOS

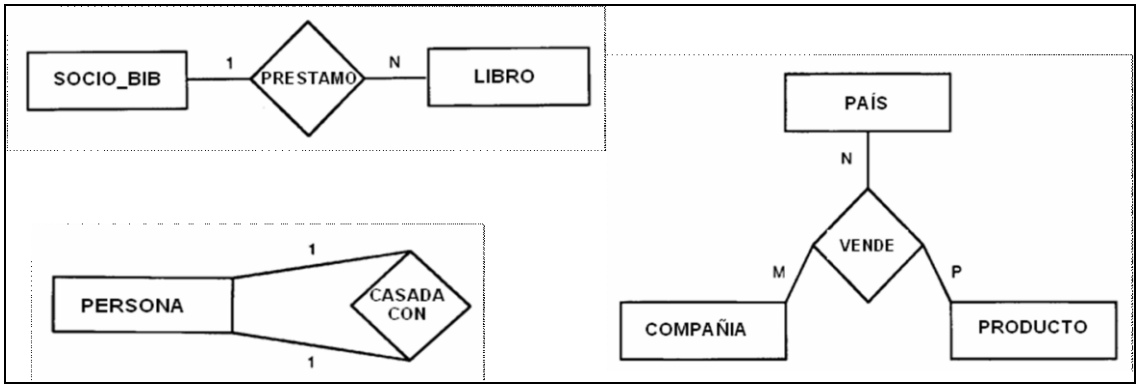


FIGURA 18: EJEMPLOS DE RELACIONES DE DISTINTOS GRADOS Y MULTIPLICIDADES

Restricciones en relaciones

- **Multiplicidad**[2], 1:1, 1:N, N:M. Establece el máximo número de entidades de un tipo que pueden vincularse con entidades de otro tipo a través de una relación. Actualmente es más común utilizar la notación Mín, Máx en cada extremo de las relaciones para indicar a la vez ambos límites, mínimo y máximo. La Figura 19 ilustra el uso de multiplicidades.
- **Dependencia de Existencia** (o Restricción de participación). Especifica si la existencia de una entidad está sujeta a que esté relacionada con otra entidad a través de un tipo de relación.
- **Entidad Débil**: Es una entidad cuya clave se forma por la clave primaria de otra entidad. Un tipo de entidad débil tendrá siempre una dependencia de existencia con respecto a otro tipo de entidad y mediante un tipo de relación de identificación.

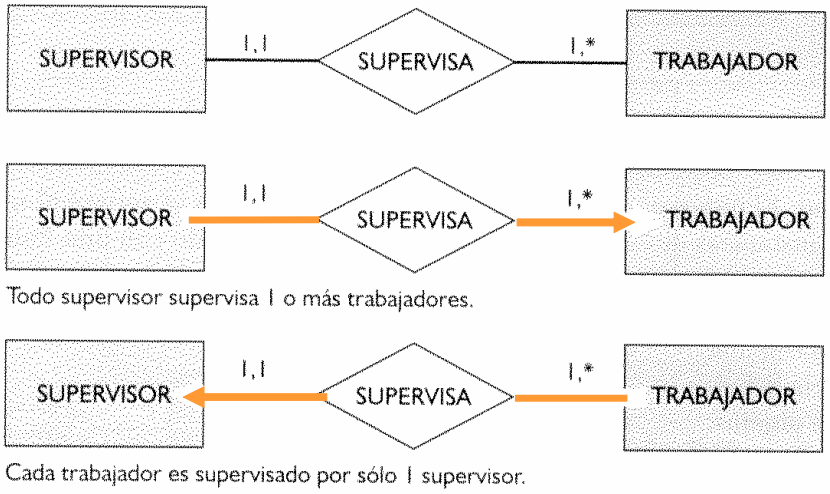


FIGURA 19: MULTIPLICIDADES EN RELACIONES

[2] Usaremos "multiplicidad" en lugar del término "cardinalidad" para ajustarnos al concepto equivalente y su denominación en UML. En UML "cardinalidad" se refiere a las instancias que en un momento específico están efectivamente referenciadas en una determinada asociación.

Existe una gran cantidad de variaciones propuestas para representar gráficamente las entidades, relaciones, atributos y multiplicidades en los Diagramas E/R. En esta sección hemos optado por utilizar una cuantas de ellas para que el lector se habitúe a esta situación. De todas formas las correspondencias entre diferentes notaciones no resultan difíciles de entender.

Ejemplo. Construir un modelo E-R para representar la siguiente información:

- Los profesores tienen un código interno, nombre, dirección, teléfono, categoría, departamento al que pertenece, asignaturas que imparten, en cada una de ellas que tienen un cierto número de créditos asignados.
- Las asignaturas tienen un código de asignatura, nombre, número de créditos de teoría y de práctica, departamento al cual está adscrita.
- Un departamento tiene un código, nombre, director y teléfono.

El siguiente Diagrama E/R de la Figura 20 es una solución para la especificación de los requisitos de datos del enunciado anterior.

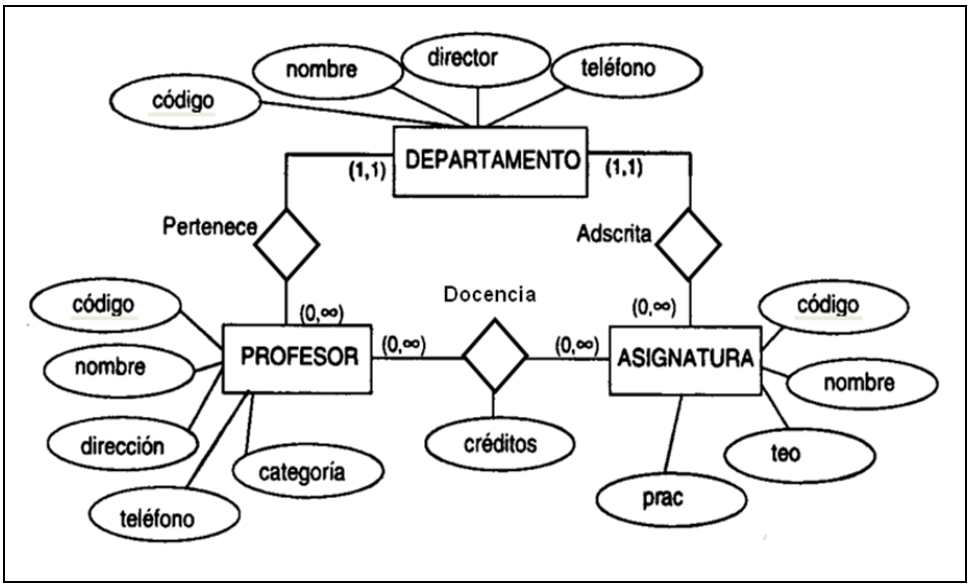


FIGURA 20: UNA SOLUCIÓN AL ENUNCIADO DEL EJEMPLO

Ejemplo. Modelar los datos asociados a la orden de compra mostrada en la Figura 21.

SERVICIO DE CONSULTORÍA
MANWARING
950 MAIN
EASTON, PA 11111

ORDEN DE COMPRA

Fecha	Número de orden	Número de vendedor
19/3	384	23

#Inventario	Descripción del producto	Precio
3821	Caja de lápices #2	4,00
4919	Caja de blocs de papel	8,90
	Impuesto	0,77
	Total	13,67

Vendedor:

Oficina Consolidada de Suministros
414 S. Choctaw Drive
Flagship, PA 12345

FIGURA 21: EJEMPLO DE ORDEN DE COMPRA

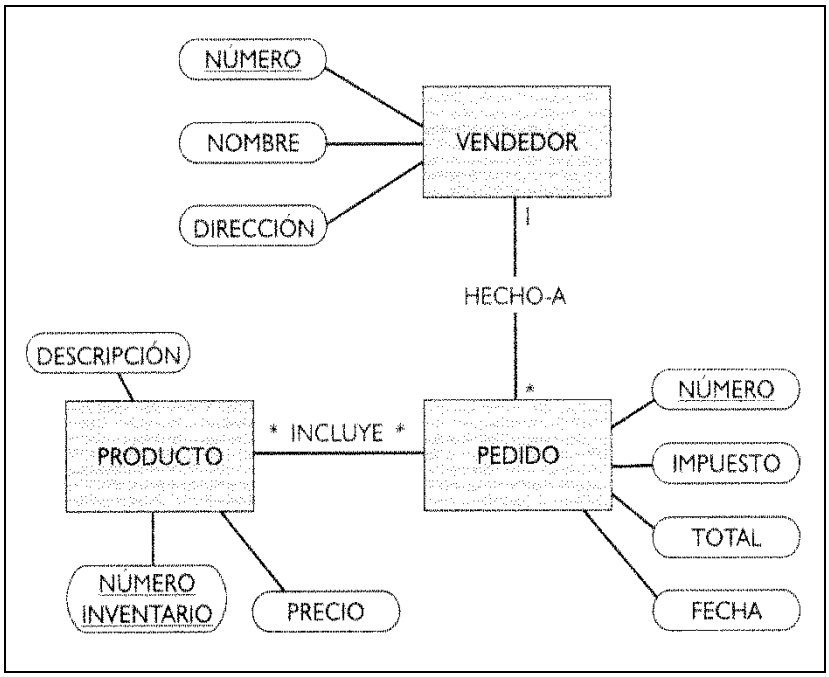


FIGURA 22: UN DIAGRAMA E/R QUE MODELA LA ORDEN DE COMPRA DE LA FIGURA 21

Generalización/Especialización. Es un mecanismo de abstracción que permite diferenciar características comunes (Generalización) y particulares (Especialización) dentro de un grupo de entidades.

Una relación de Generalización/Especialización se establece entre un tipo de entidad llamado supertipo y uno o más tipos de entidades llamados subtipos, formando jerarquías de Generalización/Especialización.

La semántica asociada es: cada entidad del subtipo es a la vez una entidad del supertipo. Esto se conoce como semántica IS-A. Todos los atributos (y relaciones) del supertipo son heredados en el subtipo. La Figura 23 muestra un ejemplo de Generalización/Especialización. Un Cliente puede se Cliente Humano o Cliente Institucional, cada uno de ellos con sus particulares atributos. Ambos tipos de cliente comparten el atributo #cliente

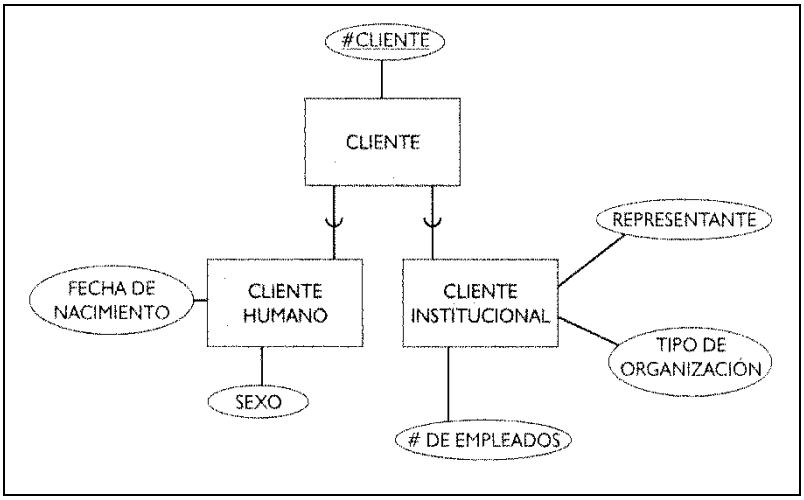


FIGURA 23: UN EJEMPLO DE GENERALIZACIÓN/ESPECIALIZACIÓN

Ejemplo. Modelar los datos asociados a la factura mostrada en la Figura 24.

SERVICIO DE CONSULTORÍA
MANWARING
950 MAIN
EASTON, PA 11111

FACTURA

Fecha	Número de factura	Proyecto
27/10	342	Sistema de control de Inventario

Consultor	Descripción de cargo	Cantidad
Rodríguez	80 horas de análisis y diseño	4.800,00
Harris	200 horas de programación	8.000,00
Farasapoulos	30 horas de entrenamiento al usuario	900,00
	Suministros (papel, fotocopias, etc.)	40,00
	Total	13.740,00

Cliente:
Sterohouse Markets
318 Elm Street
Morgantown, PA 11222

FIGURA 24: EJEMPLO DE FACTURA

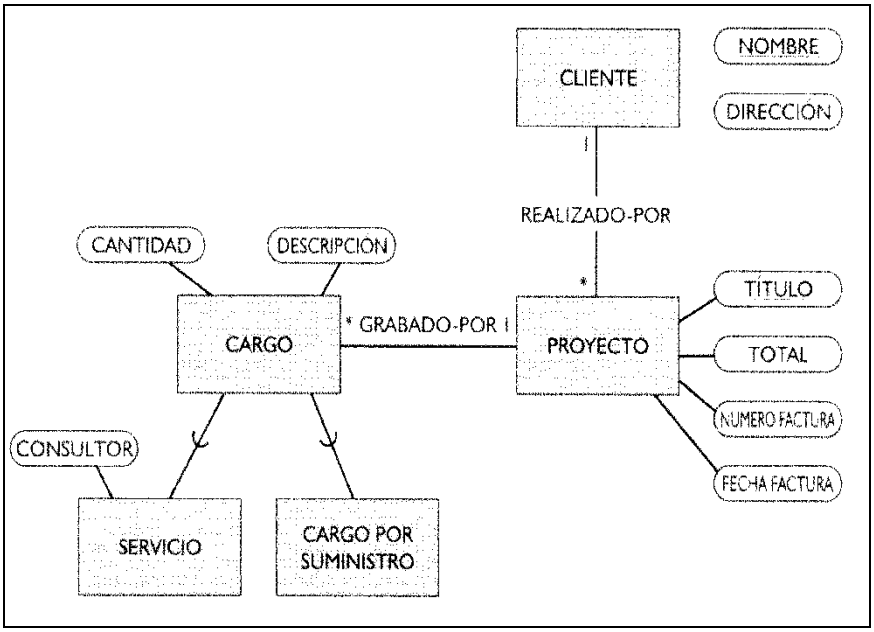


FIGURA 25: UNA SOLUCIÓN DE MODELADO DE DATOS PARA LA FACTURA DE LA FIGURA 24

Transformando un E/R en Tablas

Un Diagrama E/R puede utilizarse de forma muy simplificada con el propósito de especificar un modelo de dominio (relaciones entre conceptos del problema). Más adelante en el proceso, dicho modelo de dominio podría refinarse hasta constituir el Modelo Conceptual de datos, es decir, el modelo de datos en el ámbito de análisis. Este modelo conceptual de datos debería tener trazabilidad hacia el modelo de datos en su nivel de abstracción correspondiente al diseño lógico relacional, es decir, una especificación del esquema relacional que pueda servir para generar una implementación posterior en un determinado gestor de bases de datos relacional. A continuación, presentaremos una pautas muy simples (incluso ya están automatizadas en muchas herramientas) que permiten obtener un Modelo Lógico Relacional (o su representación gráfica como Diagramas de Tablas) a partir de un Diagrama E/R.

Estas mismas pautas pueden emplearse para obtener un Modelo Lógico Relacional a partir de un Diagrama de Clases. Para hacerlo deberías ponerte unas gafas que filtraran la información del Diagrama de Clases, ocultándote las operaciones de las clases, además de las clases y los atributos no persistentes (aquellas clases y atributos que no requieran almacenarse en la Base de Datos). Este es el simple truco para usar un Diagrama de Clases de forma similar a un Diagrama E/R.

Transformación de Entidades y Atributos

- Transformar cada entidad en una tabla. El identificador de la entidad será la clave primaria de la tabla. Si la entidad no tiene definido un identificador, para la tabla se crea un atributo adicional que se utilizará como clave primaria.

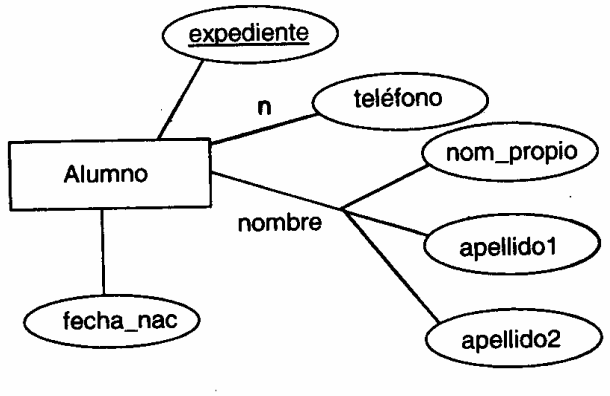
- Analizar la conveniencia de crear tablas adicionales para cada atributo multivaluado. Las nuevas tablas tendrán como clave primaria el identificador de la entidad junto al atributo multivaluado.

- Los atributos compuestos se “aplanan” dejando como atributos de la tabla sólo los componentes de más bajo nivel.

- Los atributos derivados pueden tratarse como atributos normales indicando que se trata de atributos derivados. La decisión de almacenar o no físicamente el valor del atributo derivado podría retrasarse hasta la implementación.

- Los atributos que no pueden tener valor nulo deben indicarse (salvo si son parte de la clave primaria, caso en el cual se entiende que no pueden tomar valores nulos).

La Figura 26 ilustra la obtención de tablas a partir de una entidad y sus atributos.



ALUMNO(Nro_exp, Nombre, Apellido1, Apellido2, Fec_nac)

TELEFONOS_ALUMNO(Nro_exp, Telefono)

FIGURA 26: EJEMPLO DE TRANSFORMACIÓN DE ENTIDAD Y ATRIBUTOS EN TABLAS

Transformación de Supertipos y Subtipos

Se plantean las siguientes alternativas:

I. Sólo una tabla incluyendo los atributos del supertipo más la unión de todos los atributos (y relaciones) de los subtipos.

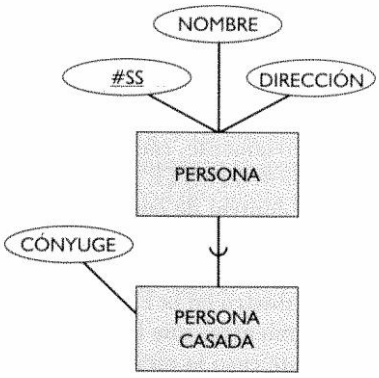
- La clave primaria es el identificador del supertipo.
- Considerando que las jerarquías son disjuntas debería establecerse una restricción de integridad que controle que los grupos de atributos tengan valores en forma excluyente unos de otros.
- Puede incluirse un atributo derivado que indique el subtipo al cual pertenece una tupla.

II. Una tabla para el supertipo y una para cada subtipo. La tabla que representa a cada subtipo tiene como atributo adicional el identificador del supertipo, pudiendo ser ésta mismo la clave primaria de la tabla del subtipo o solamente una clave ajena que referencia a la tabla del supertipo.

- Puede incluirse un atributo derivado que indique el subtipo al cual pertenece una tupla de la tabla del supertipo.

III. Sólo las tablas correspondientes a cada subtipo, incluyendo en cada una de ellas todos los atributos (y relaciones) del supertipo. Las claves primarias de las tablas de los subtipos se determinan como en el caso anterior.

Las Figuras 27 y 28 ilustran transformaciones de generalizaciones a tablas proponiendo alternativas posibles en cada caso.



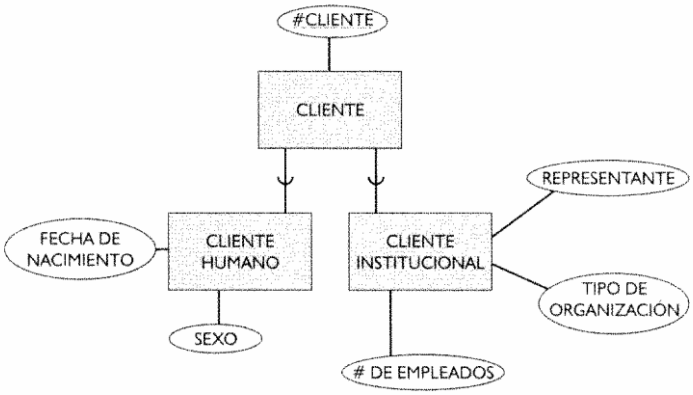
ALTERNATIVAS:

I. PERSONA(SS#, Nombre, Dirección, Cónyuge)

II. PERSONA(SS#, Nombre, Dirección)

PERSONA_CASADA(SS#, Cónyuge)

FIGURA 27: EJEMPLO DE TRANSFORMACIÓN DE GENERALIZACIÓN A TABLAS



ALTERNATIVAS:

1. CLIENTE(#Cliente, Fec_nac, Sexo, Nro_de_empleados, Representante, Tipo_org)

   RI: (Fec_nac, Sexo) no Nulo o (Nro_empleados, Representante, Tipo_org)

   No Nulo, en forma exclusiva.

2. CLIENTE(#Cliente)

   CLIENTE_HUMANO(#Cliente, Fec_nac, Sexo)

   CLIENTE_INSTITUCION(#Cliente, Nro_de_empleados, Representante, Tipo_org)

FIGURA 28: OTRO EJEMPLO DE TRANSFORMACIÓN DE GENERALIZACIÓN A TABLAS

Transformación de Relaciones

Cualquier tipo de relación podría ser tratado como un tipo de entidad (tipo de entidad asociativa) y transformarse como tal. Según esto, cada tipo de relación generaría una tabla adicional. Esto se aplicará a relaciones binarias N:M, ternarias o superiores). En el resto de relaciones binarias se recomienda dar un tratamiento especial atendiendo a las multiplicidades asociadas. La idea es reducir el número de tablas para evitar excesivo "join" en las consultas. Pero debe existir un compromiso entre dicho propósito y la evolución que pueda tener el modelo en el futuro.

Las relaciones entre entidades se representan en las tablas como claves ajenas.

Relación Binaria 1:1

Se propone analizar las siguientes alternativas:

I. Si una de las entidades no puede existir sin participar en la relación o es poco relevante (en el contexto de estudio) , incluir toda la información del tipo de entidad menos relevante (y atributos del tipo de relación, si existen) en la tabla que representa al otro tipo de entidad.

II. Incluir la clave primaria de la tabla que representa a una entidad en la tabla de la otra (será una clave ajena para ésta). Hacer lo mismo con los atributos del tipo de relación, si existen.

La Figura 29 muestra alternativas para obtener tablas a partir de una relación binaria 1:1. la Figura 30 muestra el caso para una relación binaria reflexiva 1:1

ALTERNATIVAS:

I. EMPLEADO(Atributos Empleado)

PC(Atributos PC)

PC_ASIGNADO(CP_Empleado, CP_PC, Atributos Relación)

II. EMPLEADO(Atributos Empleado, Atributos PC, Atributos Relación)

PC(Atributos PC, Atributos Empleado, Atributos Relación)

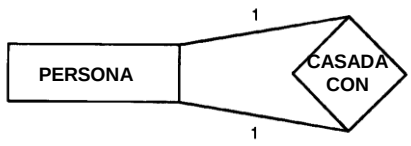
III. EMPLEADO(Atributos Empleado, CP_PC, Atributos Relación)

PC(Atributos PC)

IV. EMPLEADO(Atributos Empleado)

PC(Atributos PC, CP_Empleado, Atributos Relación)

FIGURA 29: EJEMPLO DE TRANSFORMACIÓN DE RELACIÓN 1:1



ALTERNATIVAS:

I. PERSONA(Atributos Persona, CP_Otra_Persona, Atributos Relación)

II. PERSONA(Atributos Persona)

CASADA_CON(CP_Persona, CP_Otra_Persona, Atributos Relación)

FIGURA 30: EJEMPLO DE TRANSFORMACIÓN DE RELACIÓN UNARIA 1:1

TIPO DE RELACIÓN BINARIA 1:N ("UNO A MUCHOS")

Los atributos del tipo de relación y la clave primaria de la tabla que representa al tipo de entidad de la parte "uno" se incorporan en la tabla que representa al tipo de entidad de la parte "muchos". Así, La clave primaria de la parte "uno" se convierte en una clave ajena para la tabla de la parte "muchos".

DEPTO(Atributos Departamento)

EMPLEADO(Atributos Empleado, CP_Departamento, Atributos Relación)

FIGURA 31: EJEMPLO DE TRANSFORMACIÓN DE RELACIÓN BINARIA 1:N

Relaciones Binarias N:M y Tenarias (o superiores)

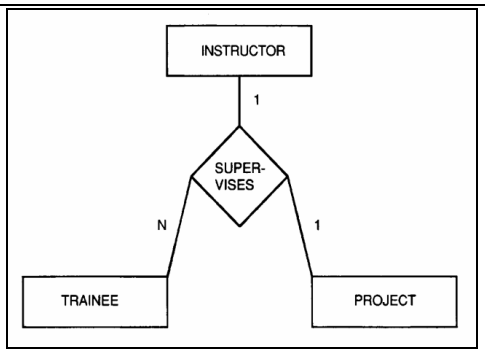
En este caso se crea una tabla para cada entidad participante y además se añade una tabla para la relación

ALUMNO(Atributos Alumno)

CURSO(Atributos Curso)

INSCRITO_EN(CP_Alumno, CP_Curso, Atributos Relación)

FIGURA 32: EJEMPLO DE TRANSFORMACIÓN DE RELACIÓN BINARIA N:M



INSTRUCTOR(Atributos Instructor)

AYUDANTE(Atributos Ayudante)

PROYECTO(Atributos Proyecto)

SUPERVISA(CP_Instructor, CP_Ayudante, CP_Proyecto, Atributos Relación)

FIGURA 33: EJEMPLO DE TRANSFORMACIÓN DE RELACIÓN TERNARIA

Integrando Modelado de Procesos y Datos

Como mencionamos anteriormente, el modelado de datos y de procesos puede hacerse en paralelo, pasando en cada caso desde análisis hasta diseño. En cada nivel de abstracción habría que asegurarse que ambos modelos son consistentes, es decir, que cada dato se procese y que estén disponibles todos los datos requeridos por los procesos. La Figura 34 ilustra el trabajo en paralelo desarrollado para el modelado de procesos y datos, enfatizando la interacción entre ambas actividades.

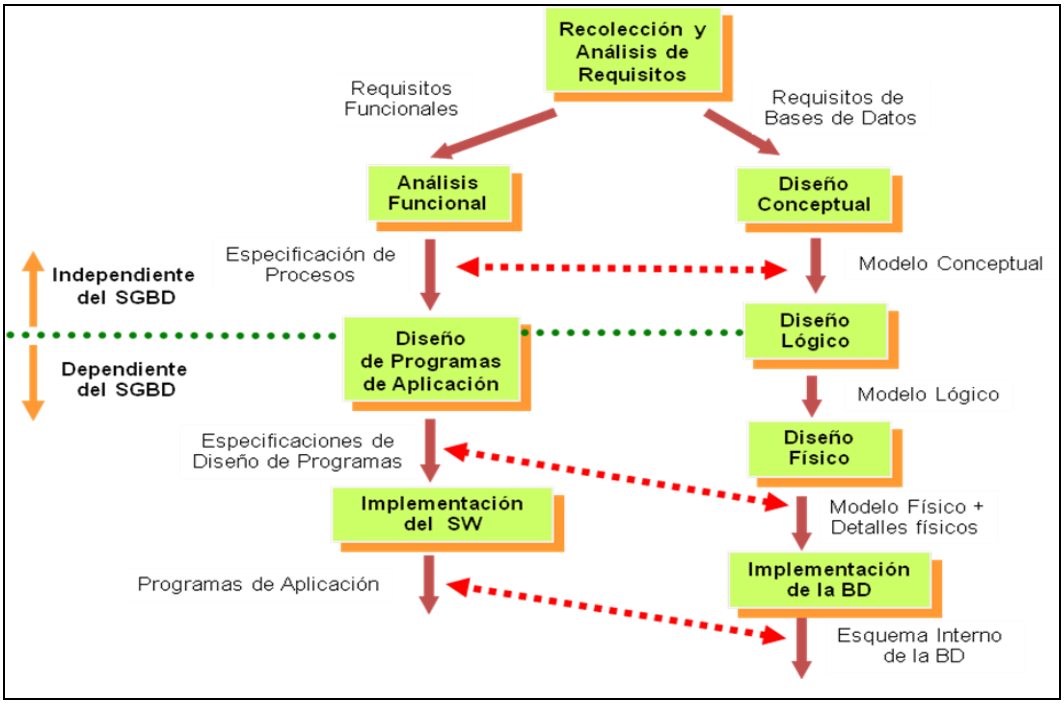


FIGURA 34: INTERACCIÓN ENTRE EL MODELADO DE PROCESOS Y EL DE DATOS

ENFOQUE ORIENTADO A OBJETOS

Ventajas del Enfoque Orientado a Objetos:

- Proximidad de los conceptos de modelado respecto de las entidades del mundo real
  - Mejora captura y validación de requisitos
  - Acerca el "espacio del problema" y el "espacio de la solución"

- Modelado integrado de propiedades estáticas y dinámicas del ámbito del problema
  - Facilita construcción, mantenimiento y reutilización

- Conceptos comunes de modelado durante el análisis, diseño e implementación
  - Facilita la transición entre distintas fases
  - Favorece el desarrollo iterativo del sistema
  - Disipa la barrera entre el "qué" y el "cómo"

¿Qué es UML?

UML es el acrónimo para Unified Modeling Language. UML es un lenguaje de propósito general para el modelado orientado a objetos. Impulsado por el Object Management Group (OMG, www.omg.org), un consorcio abierto cuyo propósito es mantener especificaciones de la industria informática que faciliten la interoperabilidad de aplicaciones. UML tiene su especificación en un documento denominado "OMG Unified Modeling Language Specification". UML combina notaciones provenientes desde:

- Modelado Orientado a Objetos
- Modelado de Datos
- Modelado de Componentes
- Modelado de Flujos de Trabajo (Workflows)

Situación de partida antes de existir UML. Diversos métodos y técnicas OO, con muchos aspectos en común, pero utilizando distintas notaciones. Inconvenientes para el aprendizaje, aplicación, construcción y uso de herramientas, etc. Pugna entre distintos enfoques (y correspondientes gurús). Era evidente que hacía falta una notación estándar.

La especificación de UML ha ido sufriendo continuas modificaciones y el documento de especificación cuenta con varias versiones. La versión actual (en agosto de 2016) es UML 2.5, documento descargable desde: http://www.omg.org/spec/UML/2.5/PDF/.

Los principales Diagramas de UML (presentes en la mayoría de herramientas de apoyo a UML) son:

- Diagrama de Casos de Uso
- Diagrama de Clases
- Diagrama de Objetos
- Diagramas de Comportamiento
  - Diagrama de Estados
  - Diagrama de Actividad
  - Diagramas de Interacción
    - Diagrama de Secuencia
    - Diagrama de Colaboración
- Diagramas de implementación
  - Diagrama de Componentes
  - Diagrama de Despliegue

El Diagrama de Objetos en realidad no se provee como un tipo de diagrama separado. En los Diagramas de Secuencia, Diagramas de Colaboración y en Diagramas de Actividad se representan objetos.

En algunos libros se refieren a Diagramas de Paquetes, Diagramas de Subsistemas y Diagramas de Modelos. Sin embargo, éstos corresponden a casos particulares de los diagramas arriba mencionados; cuando en éstos sólo se incluye paquetes (o subsistemas, o modelos, respectivamente).

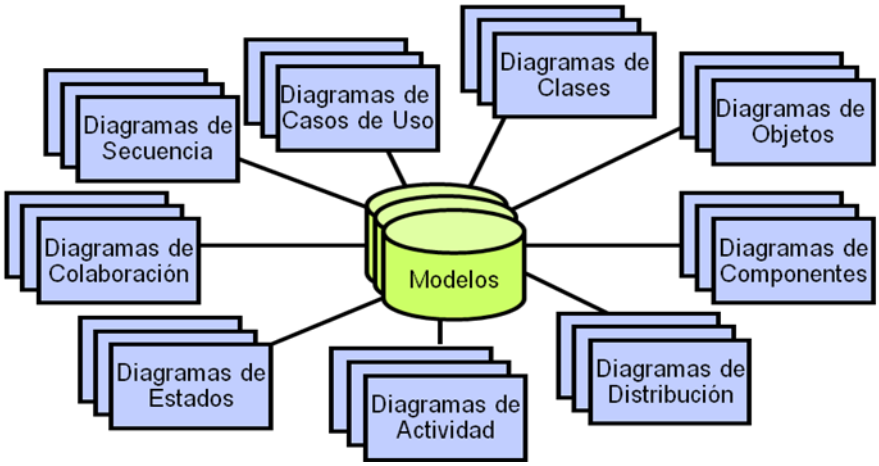


FIGURA 35: DIAGRAMAS UML PARA ESPECIFICAR MODELOS DEL SISTEMA

Como hemos comentado al principio, los diagramas expresan gráficamente partes de un modelo (esto se ilustra en la Figura 35). Así, utilizando combinaciones de diagramas UML podemos especificar los modelos que nos sean de interés. Es importante remarcar que UML es una notación, es decir, no incluye guía respecto de como utilizarlo. Por otra parte, aunque algunos diagramas son usados más frecuentemente en ciertas actividades del proceso de

desarrollo (por ejemplo, Diagramas de Casos de Uso para especificar requisitos), podríamos utilizar cualquier combinación de diagramas que nos parezca conveniente para especificar cierto modelo. Tampoco existe una secuencia preestablecida para la utilización de los diagramas, la trazabilidad entre los diagramas/modelos debe estar establecida en la metodología que estemos usando.

Breve Tour por UML

En esta sección revisaremos brevemente los diagramas de UML para proporcionar una visión global de la notación y las principales particularidades en cuanto a expresividad ofrecida por cada diagrama.

Paquetes en UML

Los paquetes ofrecen un mecanismo general para la organización de los modelos/subsistemas agrupando elementos de modelado. Se representan gráficamente como se muestra en la Figura 36.

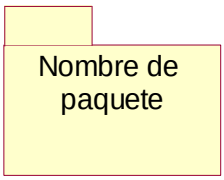


FIGURA 36: REPRESENTACIÓN DE UN PAQUETE EN UML

Cada paquete corresponde a un submodelo (subsistema) del modelo (sistema). Un paquete puede contener otros paquetes, sin límite de anidamiento, pero cada elemento pertenece a (está definido en) sólo un paquete. Una clase de un paquete puede aparecer en otro paquete por la importación a través de una relación de dependencia entre paquetes

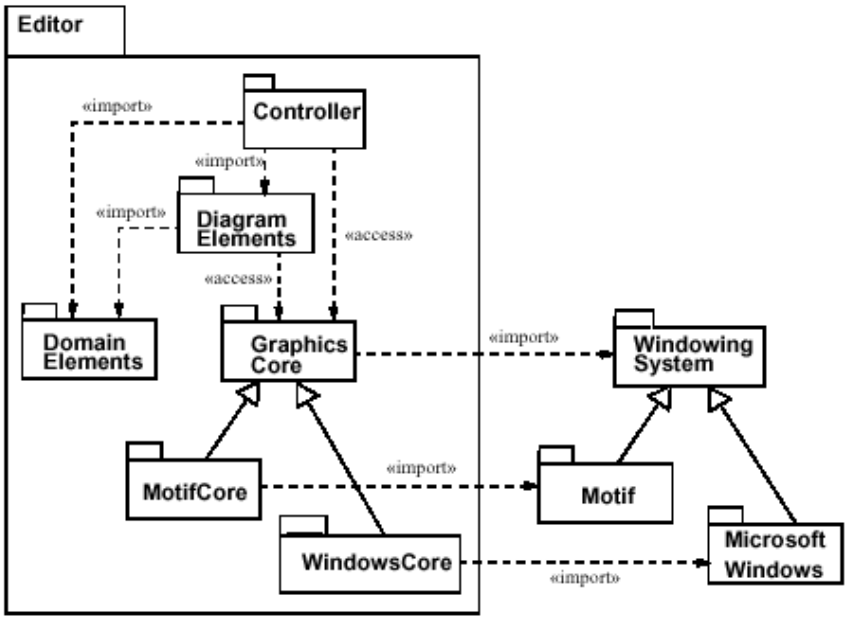


FIGURA 37: UN DIAGRAMA SÓLO MOSTRANDO RELACIONES ENTRE PAQUETES

En la Figura 37 podemos observar diferentes relaciones entre paquetes. El paquete Editor contiene varios paquetes. Las líneas punteadas son dependencias entre paquetes. las etiquetas con "<<>>" se denominan estereotipos y otorgan un significado particular a algún elemento de modelado, en este caso a las dependencias. Así, algunas dependencias de la Figura 37 representan relaciones de importación de especificaciones de un paquete a otro y relaciones de acceso (por ejemplo, invocación desde un elemento de un paquete a otro elemento en el otro paquete).

Diagrama de Casos de Uso

Casos de Uso es una técnica para capturar información respecto de los servicios que un sistema proporciona a su entorno. No pertenece estrictamente al enfoque orientado a objeto, es una técnica para captura y especificación de requisitos. Los elementos básicos de un diagrama de Casos de Uso son los Actores, los Casos de Uso y las relaciones de comunicación entre actores y Casos de Uso.

Ejemplo. La Figura 38 muestra **l**os Casos de Uso que estarían disponibles para un cliente en una aplicación de cajero automático

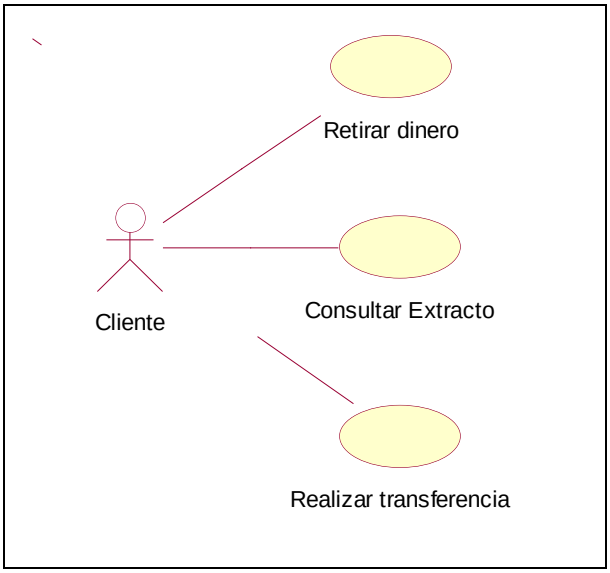


FIGURA 38: CASOS DE USO PARA EL ACTOR CLIENTE

Relaciones entre Casos de Uso. Un Caso de Uso puede tener una relación de dependencia con respecto de otro. A partir de UML 1.3 se disponen de tres tipos de relaciones entre casos de uso, representadas por un símbolo de generalización desde un caso de uso a otro. Los tipos de relación son: Inclusión (con el estereotipo <<include>>), Extensión (con el estereotipo <<extend>>) y Generalización (sin estereotipo).

No es recomendable incluir estos tipos de relaciones entre Casos de Uso en los Diagramas de Casos de Uso, especialmente cuando tengan que ser enseñados al cliente. Esto evita complicaciones innecesarias de la especificación. El momento oportuno para añadir con criterio dichas relaciones es cuando el desarrollador ya ha validado con el usuario el modelo de requisitos. Las cuestiones para plantearse son las siguientes:

Si se fuera a especificar en detalle un Caso de Uso y se prevé que varios Casos de Uso tienen parte de su especificación en común, dicha parte común puede factorizarse separándola en un nuevo caso de uso que es "invocado" desde aquellos que reutilizan dicha especificación. Este es el caso de utilización de <<include>>. La Figura 39 muestra un ejemplo de utilización de <<include>>.

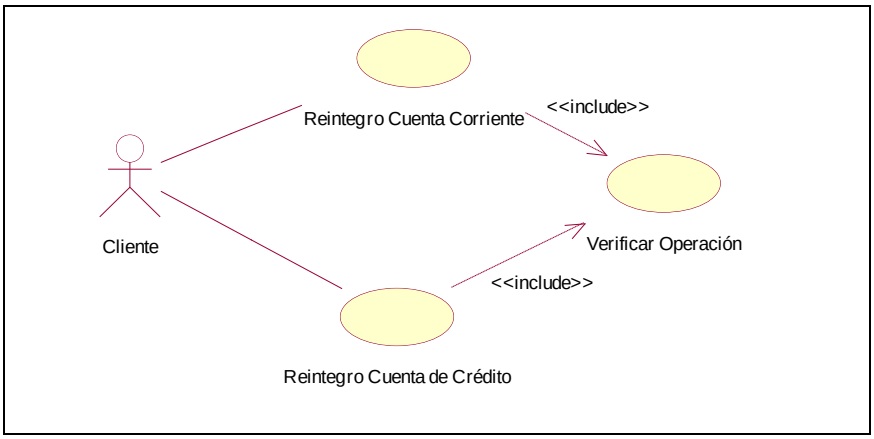


FIGURA 39: CASOS DE USO CON LA RELACIÓN DE DEPENDENCIA <<INCLUYE>>

Si se fuera a especificar en detalle un Caso de Uso y se prevé que un Caso de Uso es demasiado complejo pero que parte de dicha complejidad está asociada al cumplimiento de una condición, dicha parte puede factorizarse separándola en un nuevo caso de uso que se vuelve a "mezclar" en el original cuando se cumple la condición. Este es el caso de utilización de <<extend>>. La Figura 40 muestra un ejemplo de utilización de <<extend>>.

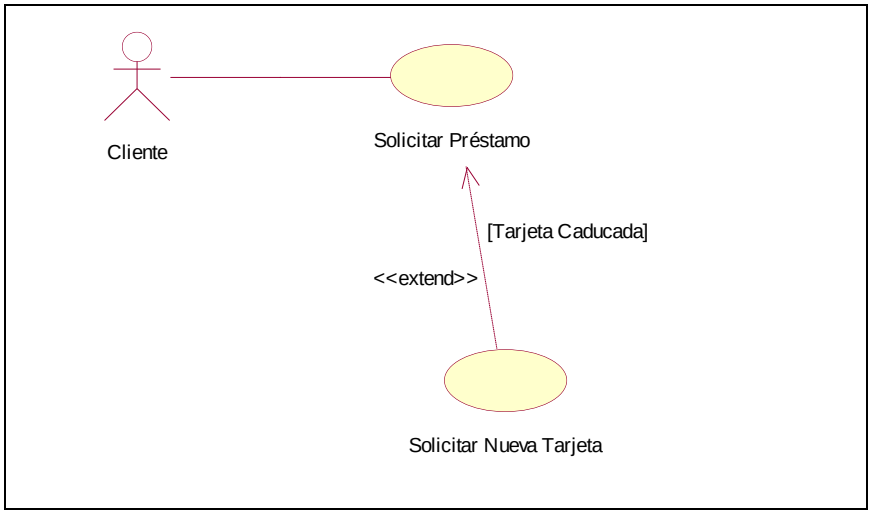


FIGURA 40: CASOS DE USO CON LA RELACIÓN DE DEPENDENCIA <<EXTENDS>>

Nótese que se hace énfasis en que sólo interesa plantearse el uso de <<incluye>> y <<extend>> cuando se pretende especificar en mayor detalle un Caso de Uso. Es importante destacar que no todos los Casos de Uso requieren el mismo grado de especificación, muchos de ellos podrían quedarse especificados sólo con el nombre del Caso de Uso.

Cada Caso de Uso puede estar definido en mayor detalle por:

- Texto que lo describe
- Secuencia de pasos (flujo de eventos) ejecutados dentro del caso de uso
- Precondiciones y postcondiciones para que el caso de uso comience o termine
- Cualquier técnica para especificar procesos (cuando el procesamiento del Caso de Uso tiende a ser algorítmico)
- Plantilla incluyendo algunas de las alternativas anteriores. Por ejemplo, una plantilla como la mostrada en la Figura 41.

Identificador	CU-<**id-requisito**>	
Nombre	<**nombre del requisito funcional**>	
Descripción	El sistema deberá comportarse tal como se describe en el siguiente caso de uso { concreto cuando <evento de activación> , abstracto durante la realización de los casos de uso <lista de casos de uso>}	
Precondición	<precondición del caso de uso>	
Secuencia	**Paso**	**Acción**
Normal	1	{El <actor> , El sistema} <acción realizada por el actor o sistema>, se realiza el caso de uso < caso de uso CU-x>
	2	Si <condición>, {el <actor> , el sistema} <acción realizada por el actor o sistema>>, se realiza el caso de uso < caso de uso CU-x>
	…	…
Postcondición	<postcondición del caso de uso>	
Excepciones	**Paso**	**Acción**
	1	Si <condición de excepción>,{el <actor> , el sistema} }<acción realizada por el actor o sistema>>, se realiza el caso de uso < caso de uso CU-x>, a continuación este caso de uso {continua, aborta}
	…	…
Rendimiento	**Paso**	**Cota de tiempo**
	1	n segundos
	…	…
Frecuencia esperada	<nº de veces> veces / <unidad de tiempo>	
Importancia	{sin importancia, importante, vital}	
Urgencia	{puede esperar, hay presión, inmediatamente}	
Comentarios	<comentarios adicionales>	

FIGURA 41: UNA POSIBLE PLANTILLA PARA ESPECIFICAR CASOS DE USO[3]

El problema con las plantillas como la de la Figura 41 es que existe demasiada tentación a rellenarlo todo y muchas veces se incluye información obvia. Además, cuando se aplica la misma forma de especificación para todos los Casos de Uso el trabajo de especificación termina siendo muy tedioso y repetitivo y se tiene el riesgo de no centrarse en los aspectos más críticos del sistema. Todos los Casos de uso no tienen la misma criticidad, con lo cual nuestro esfuerzo debemos orientarlos a especificar de manera selectiva privilegiando a los Casos de Uso de acuerdo con su relevancia en el contexto.

Diagrama de Secuencia

Los Diagramas de Secuencia y de Colaboración son usados para describir gráficamente un caso de uso o un escenario de uso del sistema. Un Diagrama de Secuencia muestra los objetos de un escenario mediante líneas verticales y los mensajes entre objetos como flechas conectando objetos. Los mensajes son dibujados cronológicamente desde arriba hacia abajo. Los rectángulos en las líneas verticales representan los periodos de actividad de los objetos. La Figura 42 muestra un ejemplo de Diagrama de Secuencia para un escenario que podríamos denominar “Préstamo de libro con éxito”.

[3] adaptada desde http://www.lsi.us.es/~amador/publicaciones/metodologia_analisis.pdf.zip

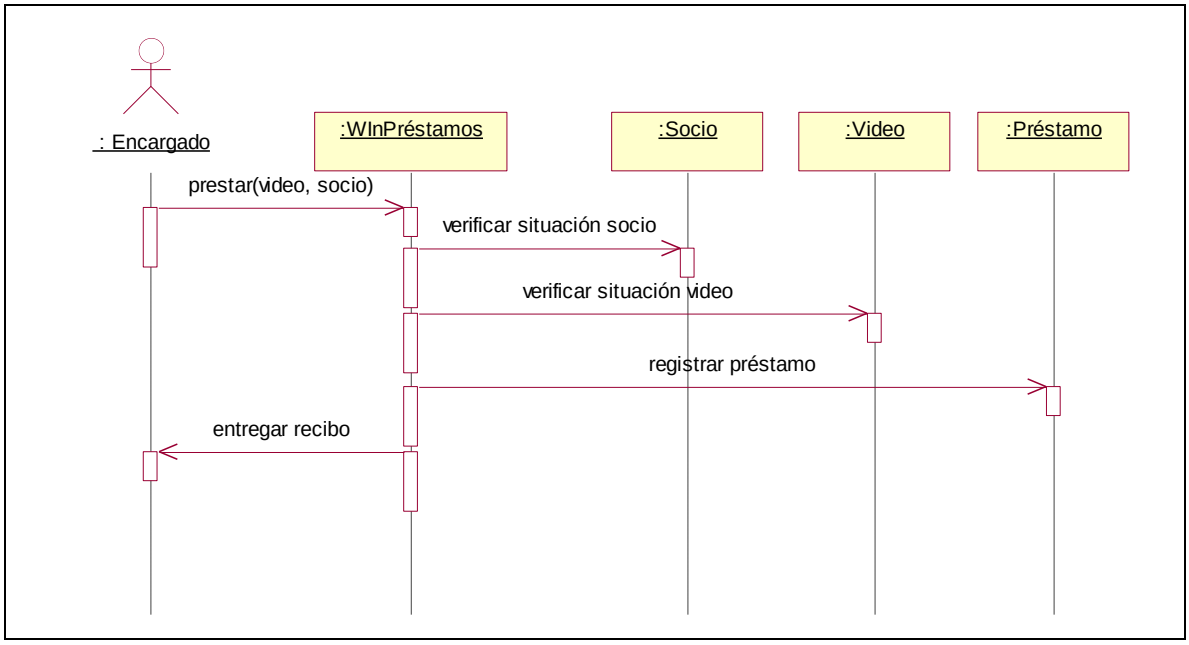


FIGURA 42: EJEMPLO DE DIAGRAMA DE SECUENCIA

Los Diagramas de Secuencia y de Colaboración son usados para describir gráficamente un caso de uso o un escenario.

Cada Caso de Uso puede tener varios escenarios de ejecución. Sin embargo, no vale la pena hacer Diagramas de Secuencia para cada escenario no para todos los Casos de Uso. Cuando estamos en el ámbito de requisitos un Diagrama de Secuencia mostraría los actores y sólo un objeto, representando a todo el sistema. Posteriormente en el ámbito de análisis un Diagrama de Secuencia podría ilustrar los objetos dentro del sistema (además de objetos de interfaz), incluyendo sus interacciones a través de mensajes. Esto podría detallarse hasta representar instancias de objetos del sistema con invocaciones a métodos desde un objeto a otro. Sin embargo, como ya se ha insistido, sólo en determinados escenarios podría resultar rentable hacer un esfuerzo adicional de especificación, por ejemplo, aquellos con interacciones complejas y/o asociados a funcionalidad crítica del sistema.

Diagrama de Colaboración

El Diagrama de Colaboración modela la interacción entre los objetos de un Caso de Uso. Los objetos están conectados por enlaces (links) en los cuales se representan los mensajes enviados acompañados de una flecha que indica su dirección. El Diagrama de Colaboración ofrece una mejor visión del escenario cuando el analista está intentando comprender la participación de un objeto en el sistema.

Los Diagramas de Secuencia y de Colaboración son vistas diferentes de la misma información, con lo cual tener las dos representaciones para un escenario es redundante. En un Diagrama de Secuencia tenemos una vista cronológica de las interacciones. En el Diagrama de Colaboración tenemos una vista espacial de las interacciones. Las herramientas CASE suelen ofrecer la generación automática de un Diagrama de Colaboración a partir de

un Diagrama de Secuencia, o viceversa. La Figura 43 ilustra un Diagrama de Colaboración correspondiente al Diagrama de Secuencia de la Figura 42.

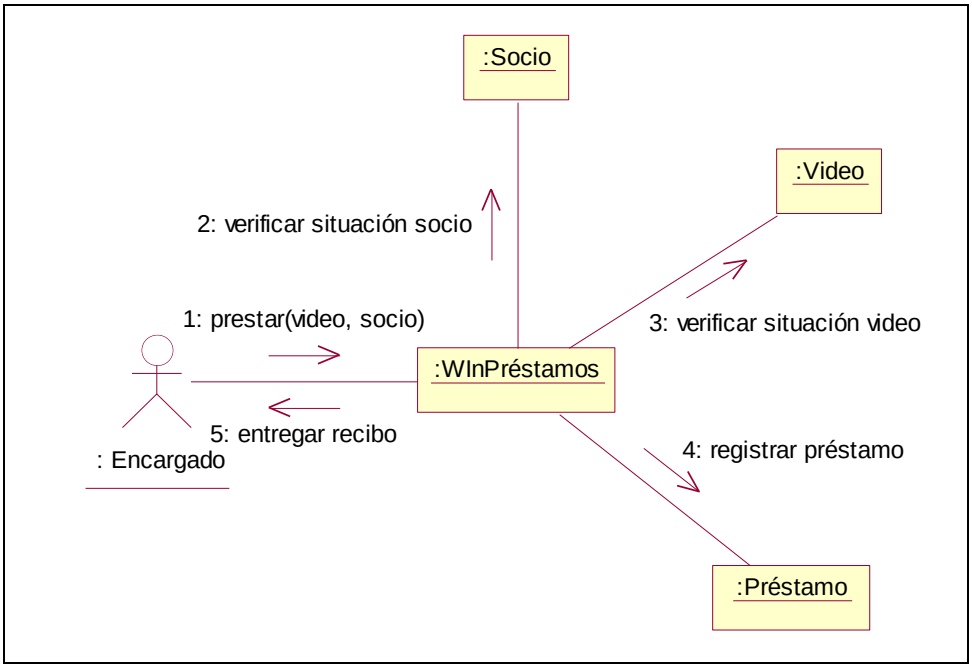


FIGURA 43: EJEMPLO DE DIAGRAMA DE COLABORACIÓN

Diagrama de Clases

El Diagrama de Clases es el diagrama principal para el análisis y diseño del sistema. Si tuviésemos que prescindir de otros diagramas, para poder decir que estamos modelando orientado a objetos, al menos deberíamos tener el Diagrama de Clases.

Un diagrama de clases presenta las clases del sistema con sus relaciones estructurales y de herencia. La definición de clase incluye especificaciones para atributos y operaciones. En cuanto a trazabilidad, el modelo de requisitos (Casos de Uso) debería aportar información para derivar de él las clases, objetos, atributos y operaciones.

La Figura 44 muestra la representación de una clase, con los compartimientos para atributos y para operaciones. Los atributos tienen una decoración para indicar que son privados (sólo modificables mediante operaciones de la clase). Las operaciones tienen una decoración para indicar que son públicos, es decir, son accesibles desde cualquier otra clase.

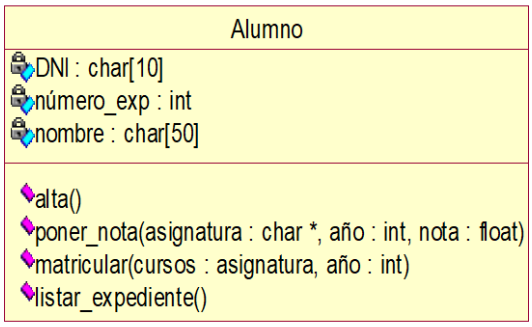


FIGURA 44: REPRESENTACIÓN DE UNA CLASE

Asociación entre clases. La Figura 45 muestra una asociación entre las clases Departamento y Profesor, se incluyen las multiplicidades y roles correspondientes. Así, por ejemplo, "un Departamento tiene un (y sólo un) Profesor director. En la Figura 46 se muestra una Clase Asociación, es decir una clase que está asociada a una asociación (valga la redundancia). Este tipo de conexión es imprescindible cuando se tiene una asociación muchos-a-muchos y las propiedades son del concepto asociación más que las clases participantes. Esta situación también ocurre en los Diagramas E/R y se modela de forma similar, bien poniendo directamente atributos a la relación o introduciendo una Entidad Asociativa.

FIGURA 45: ASOCIACIÓN ENTRE CLASES

Generalización/Especialización. En la Figura 47 se muestra la especialización de la clase Trabajador en tres subclases: Directivo, Administrativo y Obrero. Además, la jerarquía se ha "decorado" con la restricción de integridad (texto entre { }) disjunta, completa. Disjunta quiere decir que un trabajador, en un momento dado sólo puede pertenecer a una subclase. Completa significa que todos los trabajadores pertenecen a alguna de las subclases.

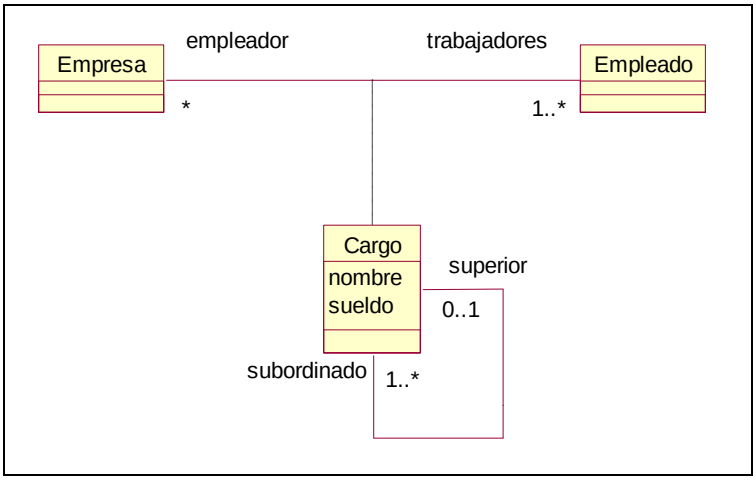


FIGURE 46: EJEMPLO DE CLASE ASOCIACIÓN

Toda jerarquía de generalización/especialización debe tener una justificación práctica basada en la conveniencia de especificar propiedades comunes en la clase padre y las propiedades específicas en las clases hijas. Considerad que también las asociaciones de una clase se tratan como si fueran atributos de la clase, es decir, también se heredan desde la clase padre a las hijas.

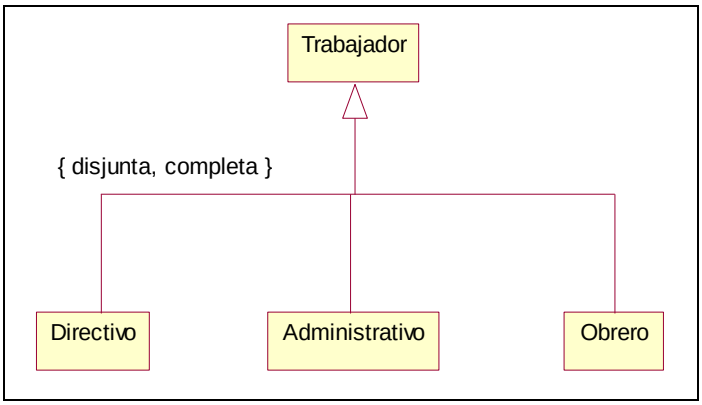


FIGURE 47: EJEMPLO DE JERARQUÍA DE GENERALIZACIÓN/ESPECIALIZACIÓN

La Generalización y la Especialización son caras de la misma moneda. La diferencia está en el orden de generación que se ha seguido. Si se tiene una clase y posteriormente se definen subclases debido a que se requiere especificar propiedades específicas, entonces estaremos "especializando". Si por el contrario, se tienen varias clases y se detecta que tienen propiedades comunes que se ponen en una nueva clase padre, entonces estamos “generalizando”. Una vez creada la jerarquía lo relevante es que disponemos del mecanismo de herencia el cual hace que en las clases hijas estén disponibles (se heredan) las especificaciones de la clase padre. Esto es un potente mecanismo que organizar de mejor forma las propiedades, ofreciendo reutilización y extensibilidad de dichas especificaciones, basándose en el concepto de clase.

La Figura 48 muestra un ejemplo en el cual se combinan algunos elementos de notación disponibles en los Diagramas de Clases. La letra “n” es equivalente a poner un “*” para indicar

que no hay restricción de mínima o máxima multiplicidad. El rombo relleno da un matiz a la asociación, enfatizando que los motores están contenidos en los aviones, es decir, no pueden existir por sí solos (lo cual ya se indica con la multiplicidad 1 en el extremo Avión).

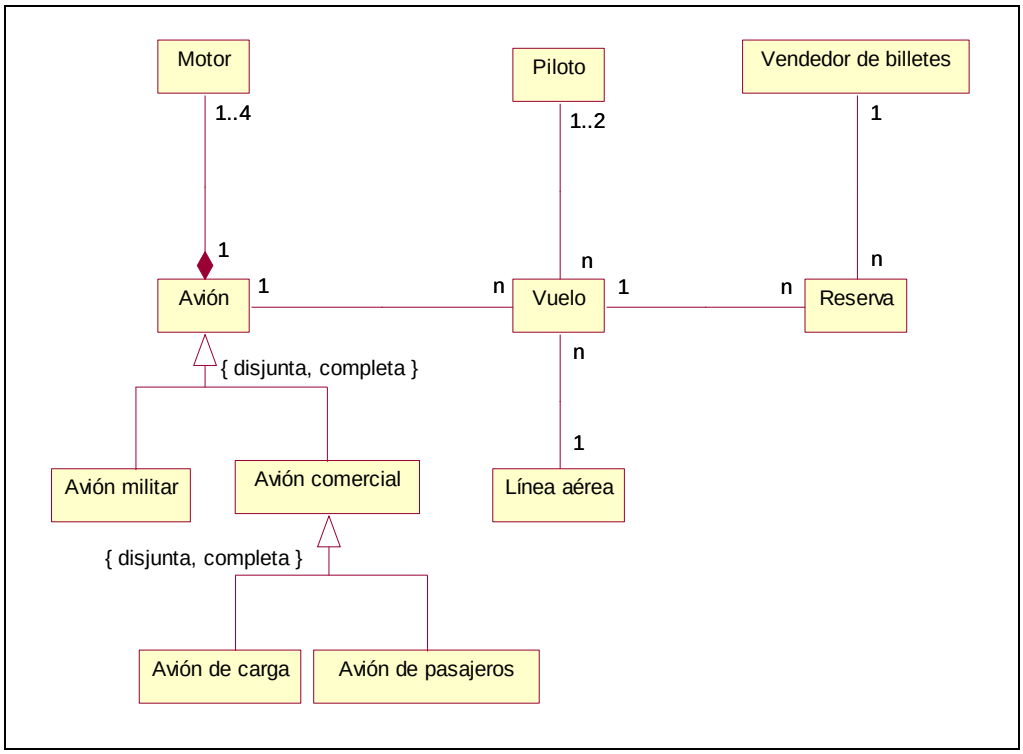


FIGURE 48: DIAGRAMA DE CLASES COMBINANDO NOTACIÓN DISPONIBLE

Es importante destacar que todas las “decoraciones” de los diagramas UML deben conllevar una trazabilidad y efecto en otros modelos o implementaciones derivadas de los modelos, de lo contrario no se están utilizando correctamente o simplemente no son necesarias. Por ejemplo en la Figura 48 que un Vuelo tenga como mínimo 1 piloto implicará que siempre los pilotos se crean antes de dar de alta los vuelos (o incluso, en extremo en la misma transacción de creación del Vuelo). Por ejemplo, si la especialización de Avión Comercial es “completa”, al momento de dar de alta o modificar un Avión Comercial siempre habría que establecer si es un Avión de Carga o Avión de Pasajeros, además por ser disjunta, sólo podría ser de un tipo. En resumen, la recomendación es no incluir en los modelos elementos que no vayan a tener una trazabilidad hacia otros modelos o finalmente en la implementación del sistema.

Diagrama de Estados

El Diagrama de Estados modela el comportamiento de una parte del sistema. Típicamente se elabora un diagrama de Estados para cada clase que tenga un comportamiento significativo. Entendemos por comportamiento significativo aquel en cual existen restricciones en cuanto a orden en el cual se pueden producir los eventos que le ocurren a un objeto, particularmente, el orden en el cual pueden darse las invocaciones a sus métodos. El comportamiento es modelado en términos del estado en el cual se encuentra el objeto, qué acciones se ejecutan en cada estado y cuál es el estado al que llega el objeto después de un determinado evento. Los eventos pueden ser ejecución de operaciones, satisfacción de condiciones o particularmente condiciones temporales (patrones de tiempo). La Figura 49 muestra el Diagrama de Estados para la clase Socio.

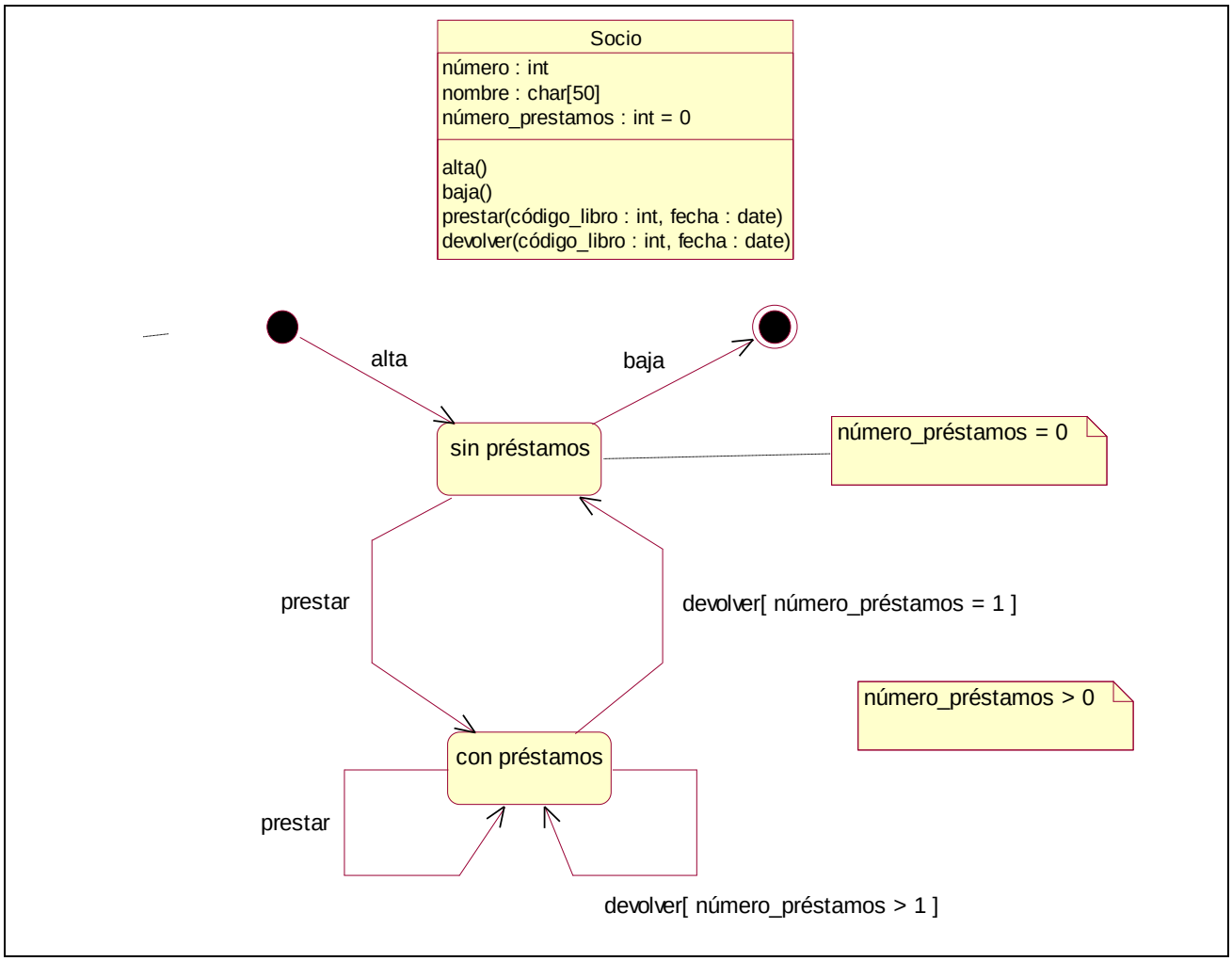


FIGURE 49: EJEMPLO DE DIAGRAMA DE ESTADOS PARA UNA CLASE

Si una clase no tiene un comportamiento significativo, su Diagrama de Estados sería simplemente el evento de creación, el estado de existencia (donde puede ocurrir cualquier evento sin restricción) y posiblemente el evento de destrucción del objeto. Esto confirma que no vale la pena hacer Diagramas de Clases den dichos casos. Tampoco vale la pena si la restricción que queremos establecer se puede especificar fácilmente con precondiciones, por ejemplo, "para retirar dinero el saldo tiene que ser mayor que 0", para lo cual no vale la pena hacer un diagrama con los estados “saldo mayor que 0” y “saldo igual a cero”.

Diagrama de Actividad

Es un caso especial de Diagrama de Estados donde:

- Todos (o la mayoría de) los estados son estados de acción
- Todas (o la mayoría de) las transiciones son “disparadas” como consecuencia de la finalización de una acción.

El Diagrama de Actividades puede utilizarse para especificar:

- El comportamiento de los objetos de una clase
- La lógica de una operación (método)
- Parte o toda la descripción de un Caso de uso
- La descripción de un Flujo de Trabajo

Un Diagrama de Actividad puede verse como una extensión de los clásicos Diagramas de Flujo, al cual se le añaden constructores tales como el de ejecución en paralelo o puntos de sincronización. Así, un Diagrama de Actividad, se suma a las técnicas para especificar procesos. La Figura 50 muestra un ejemplo de Diagrama de Actividad que especifica el algoritmo seguido por un objeto “persona” respecto de su operación “beber”. La Figura 51 ilustra un posible Flujo de Trabajo para vender un billete de avión.

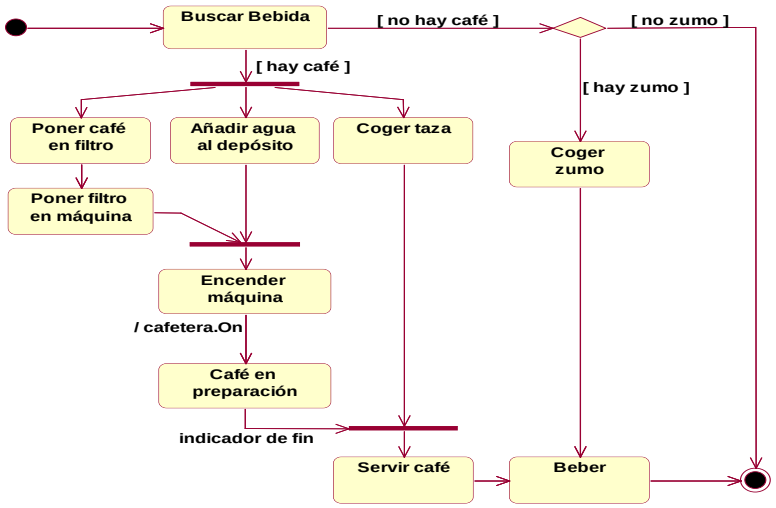


FIGURE 50: DIAGRAMA DE ACTIVIDAD PARA REPRESENTAR LA OPERACIÓN "BEBER"

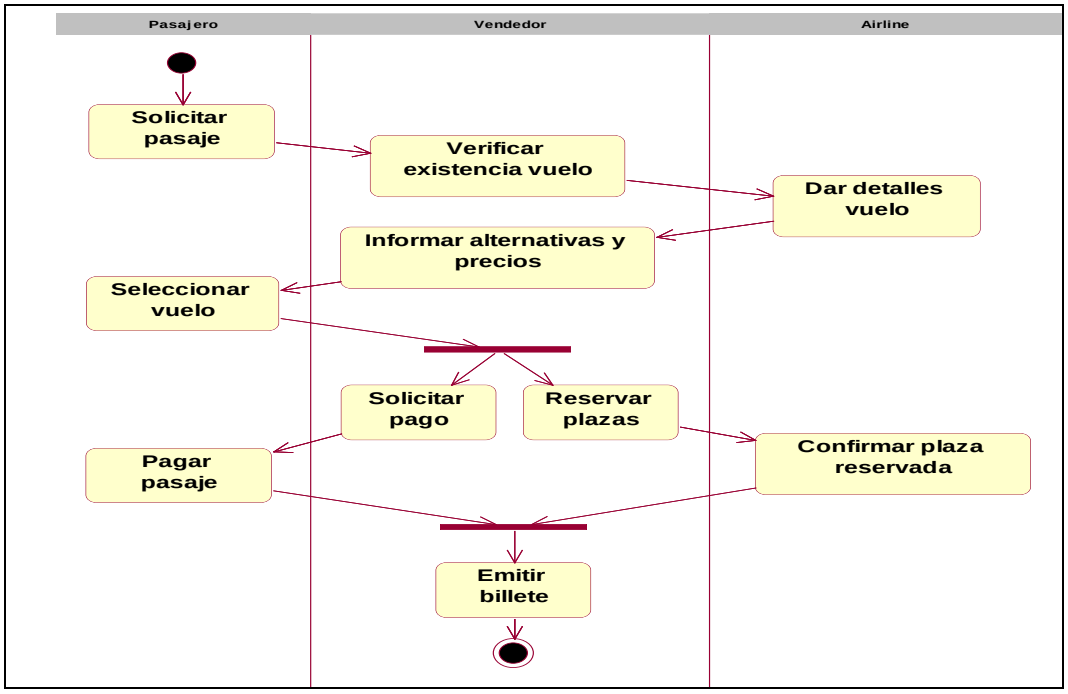


FIGURE 51: EJEMPLO DE DIAGRAMA DE ACTIVIDAD PARA ESPECIFICAR UN FLUJO DE TRABAJO

Diagrama Componentes

Un Diagrama de Componentes permite modelar la estructura del software y la dependencia entre componentes. Un componente es un grupo de clases que trabajan estrechamente. Los componentes pueden corresponder código fuente, binario o ejecutable. Una relación de dependencia indica que un componente utiliza otro, por lo cual depende de él. La Figura 52 muestra un ejemplo de Diagrama de Componentes.

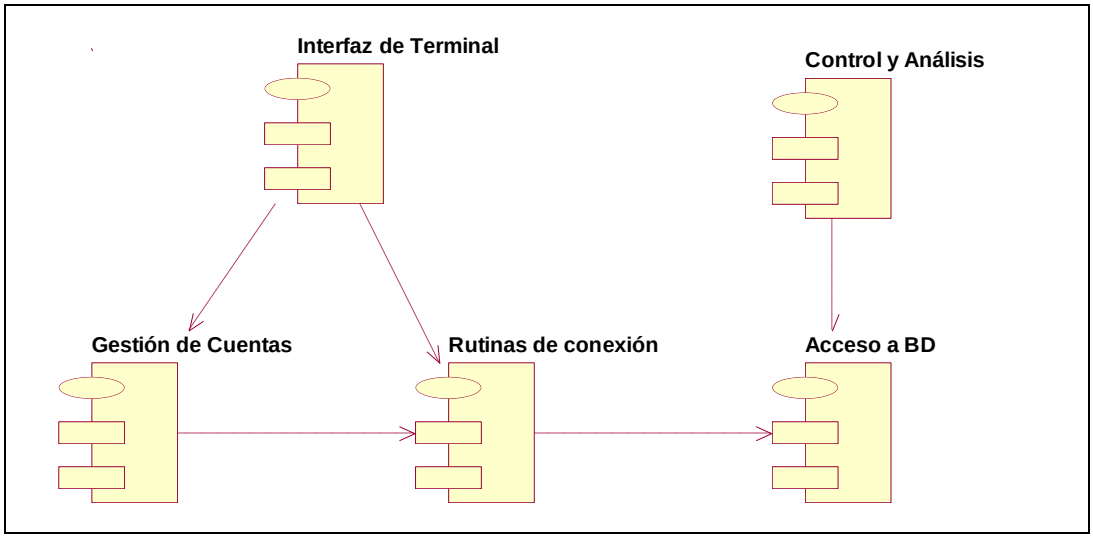


FIGURE 52: EJEMPLO DE DIAGRAMA DE COMPONENTES

El Desarrollo de Software Basado en Componentes debería ser una estrategia siempre presente en un proyecto, aunque no necesariamente todo el proceso esté exclusivamente basado en integración de componentes. Los componentes pueden haber sido desarrollados en proyectos anteriores o bien adquiridos (gratuitamente o pagando por ellos). El nivel de reutilización que proveen los componentes es de mayor granularidad que la reutilización de clases. Por esto es también importante que en lo posible el código nuevo que se desarrolle se implemente como componente (cuando tenga posibilidades de ser reutilizado).

Diagrama de Despliegue

El Diagrama de Despliegue modela la distribución en tiempo de ejecución de los elementos de procesamiento y componentes de software, junto a los procesos y objetos asociados. En el Diagrama de Despliegue se modelan los nodos y la comunicación entre ellos. Cada nodo puede contiene instancias de componentes. La Figura 53 muestra un ejemplo de Diagrama de Despliegue.

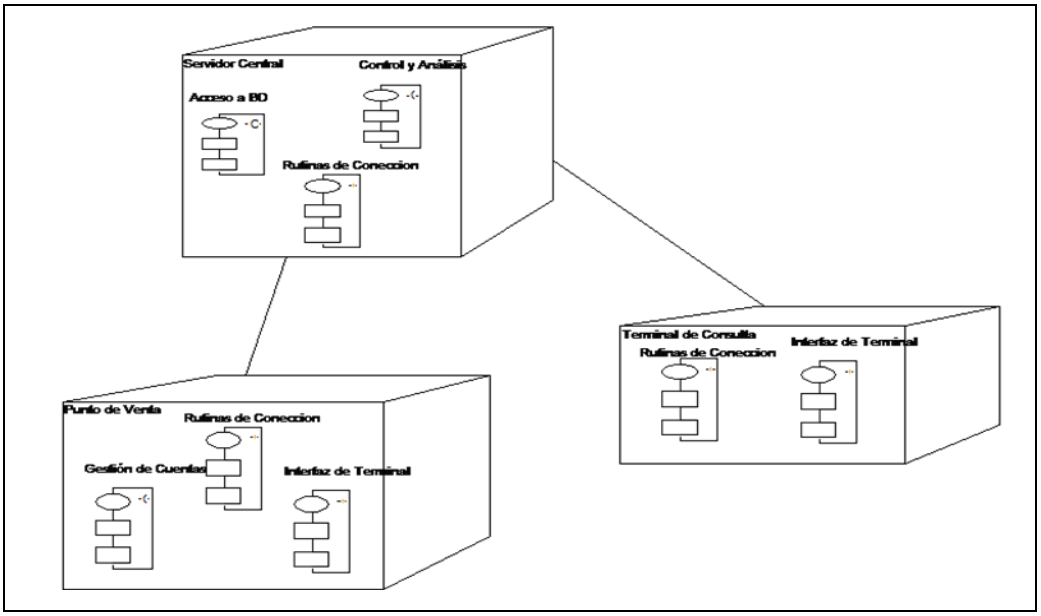


FIGURE 53: EJEMPLO DE DIAGRAMA DE DESPLIEGUE

Contrario a lo que podría interpretarse de la denominación "Diagramas de Componentes", los Diagramas de Componentes y Diagrama de Despliegue deberían elaborarse muy tempranamente en el proceso de desarrollo. Una de las primeras decisiones que debe tomarse es respecto a los componentes que se desarrollarán y los que se cogerán ya desarrollados, los cuales sólo deberán integrarse. El contar con componentes ya desarrollados y con satisfactorios niveles de calidad puede tener un gran impacto en la productividad y calidad del sistema. Asimismo, el Diagrama de Despliegue permite prever escenarios de implantación que requerirá el futuro sistema, lo cual puede crítico para asegurar el éxito final del sistema.

Comentarios respecto de los Diagramas de UML

UML es la notación más amplia/exhaustiva/extensa para modelado de software existente en la actualidad. Ha aglutinado toda la potencia de expresividad de otras notaciones previas. Tiene el respaldo de empresas de desarrollo, el mundo académico y los fabricantes de herramientas. Con todo esto, aunque UML por sí sólo esté lejos de ser la solución a los desafíos del desarrollo de software, es indudable que al menos como notación tiene garantizada una larga existencia.

Los principales vacíos que tenemos que enfrentar al utilizar UML es que la relación entre los modelos que elaboremos usando diagramas UML debemos tenerla resuelta nosotros como usuarios de la notación. Es la metodología de desarrollo la que tendría que guiarnos en cuanto a qué modelos/diagramas hacer en qué momento del proyecto, cómo refinarlos y qué consideraciones de trazabilidad entre modelos/diagramas deberíamos mantener. Este vacío se hace evidente al abrir un nuevo proyecto en una herramienta que provee soporte para UML, pues nos encontramos ante un potente editor de diagramas UML pero que no nos orienta en absoluto respecto a qué pasos debemos seguir.

“El 80 por ciento de la mayoría de los problemas pueden modelarse usando alrededor del 20 por ciento de UML” (Grady Booch). Esto quiere decir, por ejemplo, que en ciertos proyectos

podría bastar con usar Diagramas de Casos de Uso y Diagramas de Clases (con la notación básica en cada caso), sin tener "remordimientos" por no utilizar más notación de UML.

Modelado de Datos en el enfoque OO

En un "mundo OO ideal" no sería necesario el modelado de datos. Los objetos se crearían y existirían mientras no se destruyan. La persistencia de un objeto sería transparente para el programador. El lenguaje de programación o el sistema gestor de BD orientado a objetos se encargaría de persistir los objetos. En el "mundo OO actual" el uso de BD relacionales conlleva sacrificios e incongruencias en la implementación. El programador escribe sentencias SQL en los métodos y explícitamente materializa (recupera el estado de los objetos desde tablas de la BD) o desmaterializa objetos (actualiza las correspondientes tablas). De momento los frameworks de persistencia (p.e. Hibernate y ADO .NET Entity Framework) constituyen una mejora a esta situación.

Tal como ya hemos comentado. Una vez que tenemos el Modelo de Análisis/Diseño con uno o más Diagramas de Clases, si vamos a implementar en una BD Relacional, debemos obtener un Modelo Lógico Relacional (MLR) de forma similar a como se haría en el enfoque estructurado a partir de un Modelo Conceptual (representado por un Diagrama E/R). Es decir, "ver" los Diagramas de Clases como Diagramas E-R, considerando sólo las clases persistentes y sus atributos persistentes. Derivar el MLR a partir de esta "visualización". Las pautas de derivación son similares a las usadas para obtener un MLR a partir de un Diagrama E-R.

¿Qué notación y herramienta utilizar para representar gráficamente el Modelo de Datos cuando trabajamos con UML? Podríamos utilizar un Profile UML[4] para modelado de datos si la herramienta CASE ofrece soporte para profiles. Sin embargo, siendo prácticos, se recomienda que directamente se modele e implemente la BD a través de un editor de diagramas de tablas que suelen proveer los actuales sistemas gestores de BD.

[4] Un profile UML es una definición de estereotipos y restricciones específicas de modelado que permiten ajustar los diagramas de UML para modelar aspectos específicos más allá de los propuestos por UML estándar. Por ejemplo, en un profile UML para modelado de datos podríamos disponer de clases para representar tablas, asociaciones para representar claves ajenas y otros estereotipos para representar por ejemplo claves primarias, índices, etc.

Modelado de Requisitos

Un Modelo de Requisitos expresa el producto desde una perspectiva externa (caja negra). En el enfoque OO usando UML, claramente el Diagrama de Casos de Uso complementado con otros diagramas (por ejemplo, Diagrama de Clases, Diagrama de Estado, de Diagrama de Actividad) permite modelar requisitos[5].

En Modelo de Requisitos son tan importantes los requisitos funcionales (servicios que ofrece el sistema) como los requisitos NO funcionales (restricciones impuestas a los servicios ofrecidos). Ni las notaciones del enfoque estructurado ni UML abordan la especificación de requisitos no funcionales, es decir, no ofrecen notaciones gráficas para representar requisitos NO funcionales. Este vacío obliga a complementar los diagramas utilizados con especificaciones textuales para expresar los requisitos NO funcionales. La Figura 54 muestra el modelo de calidad del software propuesto por el estándar ISO/IEC 9126. En este modelo se establecen características y sub-carasterísticas que pueden usarse para evaluar la calidad del software. Es importante destacar que la característica Funcionalidad (y específicamente la sub-característica Aptitud) son las que se relacionarían con los requisitos funcionales. Todo el resto de características y sub-características se asociarían a requisitos NO funcionales del software.

[5] En el enfoque estructurado la parte alta de la jerarquía de DFDs (p.e. DFD 0) podrían ser procesos que correspondan a servicios del sistema (no a una descomposición funcional interna de ellos). Un Diagrama E-R muy general puede complementar dicho DFD en cuanto a datos.

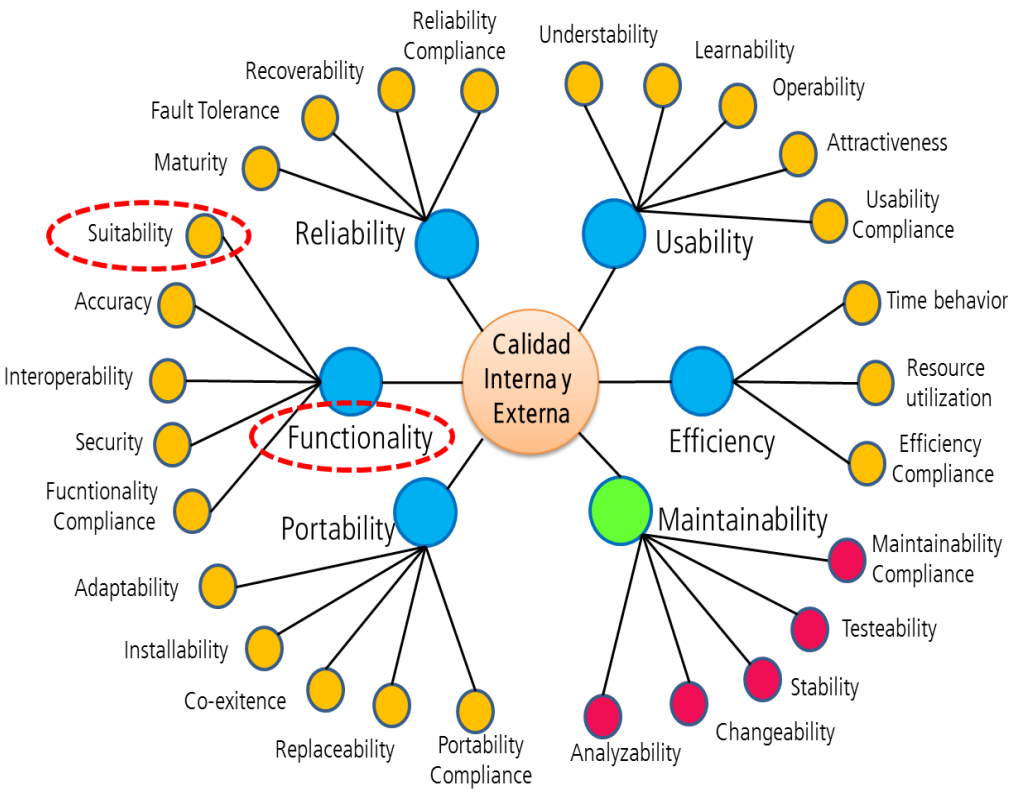


FIGURE 54: CARACTERÍSTICAS DEL SOFTWARE SEGÚN EL MODELO ISO/IEC 9126 (ACTUAL ISO 25000).

MODELADO DE SI: REFLEXIONES FINALES

¿Cuál es el propósito de nuestros modelos?

- "Documentar" (a posteriori)
- Comunicar ideas y estudiar alternativas
- Tomar decisiones de análisis/diseño que dirijan la implementación
- Generar parcial o totalmente una implementación a partir de los modelos

Hay que insistir en que los modelos deben ser "rentables", es decir, su aprovechamiento debe ser suficiente para que compense el esfuerzo invertido en elaborar y mantener los modelos.

No olvidar un principio básico: "Sencillez y Elegancia" cuando se modelo. Los modelos más complejos no son necesariamente los más útiles, sino todo lo contrario.

Gestión de modelos: Los modelos deben ser creados y mantenidos de forma colaborativa con adecuadas restricciones de acceso y control de versiones.

Trazabilidad entre modelos: debe existir un seguimiento de transformaciones de modelos hasta la implementación. Esto puede conllevar un esfuerzo significativo con lo cual debe

configurarse previamente la forma cómo se utilizarán los modelos. Por ejemplo, pueden ser en extremo modelos desechables, que sólo se usan para generar nuevos modelos hasta llegar a la implementación. Otra alternativa sería mantener actualizados y sincronizados sólo ciertos modelos.

Aprender una notación de modelado y alguna herramienta que la soporte es relativamente sencillo. Lo verdaderamente difícil es la asimilación de un proceso de desarrollo (metodología) conocido y aceptado por el equipo de desarrollo y que orqueste la explotación de herramientas y notaciones en la realización de las actividades del proceso de desarrollo.

La tendencia en ingeniería de software es clara: "generación de código a partir de modelos". Pero ¿cuáles modelos/notaciones? En principio UML o notaciones muy cercanas a UML, Pero para los que piden más algunos puntos de referencia al respecto son los siguientes:

- Extendiendo UML mediante Profiles ("más allá de UML estándar")
- Modelado para Dominios Específicos, Domain-Specific Modeling (DSM, www.dsmforum.org). Tecnología en desarrollo: Eclipse Modeling Framework (EMF) y Microsoft Tools for Domain Specific Languagues (DSL Tools).
- Meta CASE Tools (o "construye tu propia notación y herramienta"). Por ejemplo, MetaEdit+ (www.metacase.com)